Lattice-Boltzmann Methods for Cardiac Simulation

Part IV of Cardiac Computational Modeling

From the Heat Equation to Monodomain and Bidomain Models

A Feynman-Style Introduction

Contents

1. Why LBM? — Discovering a New Paradigm Through the Heat Equation
   - 1.1 The Heat Equation Revisited: A Hidden Redistribution
   - 1.2 From Weighted Averages to Distribution Functions
   - 1.3 The Direction Matrix: Lattice Geometry
   - 1.4 Collision: The Art of Local Relaxation
   - 1.5 Averaging and Flux Conservation
   - 1.6 The Eigen-Angle Matrix: Anisotropic Diffusion
   - 1.7 The Transformation Matrix M: From Distributions to Moments
   - 1.8 The Chapman–Enskog Bridge: From Lattice to PDE
   - 1.9 Boundary Conditions: Bounce-Back
   - 1.10 From Heat Equation to Cardiac Diffusion
2. Lattice-Boltzmann Methods for Monodomain
   - 2.1 From PDEs to Particle Distributions
   - 2.2 The D2Q5 and D3Q7 Lattices
   - 2.3 BGK Collision and Streaming
   - 2.4 MRT Collision: Anisotropic Diffusion
   - 2.5 Bounce-Back Boundary Conditions
   - 2.6 The Complete LBM-EP Timestep
   - 2.7 LBM vs. Classical Methods
   - 2.8 GPU Acceleration
3. Lattice-Boltzmann Methods for Bidomain
   - 3.1 The Bidomain Challenge: One Parabolic + One Elliptic
   - 3.2 Strategy 1: Pseudo-Time Relaxation
   - 3.3 Strategy 2: Hybrid LBM–Classical Solver
   - 3.4 The Dual-Lattice Architecture
   - 3.5 Performance and Implementation Outlook

Appendix

1. **A.** A Visual Guide to Differential Equations

PART IV

Lattice-Boltzmann Methods

A radically different paradigm for solving diffusion equations — no matrices, no linear solves, just fictitious particles streaming and colliding on a lattice. From the heat equation to cardiac monodomain and bidomain, LBM trades algebraic machinery for embarrassingly parallel data movement.

CHAPTER 17

The Lattice-Boltzmann Method: From Kinetic Theory to Computation

In Chapters 13–16, we discretized partial differential equations. We started with physics at the macroscopic scale — fluid flow, heat diffusion, electrical signal propagation — and asked: how do we approximate the derivatives on a computer? Those methods (FDM, FEM, FVM) work brilliantly and are the industrial standard. But there is another path, one that begins not with a PDE, but with something more fundamental: the motion of individual particles, governed by kinetic theory. This chapter introduces the Lattice-Boltzmann Method (LBM). Unlike classical discretizations that approximate a PDE, LBM simulates a discrete analogue of the Boltzmann equation — a statement about how particles move and collide. The miracle is that when you correctly simulate the microscopic collisions, the macroscopic behavior (flow, diffusion, electrical conduction) emerges automatically. For cardiac problems, especially when spatial resolution is tight or parallelization is critical, LBM offers a compelling alternative to classical methods. We will build this from first principles: the Boltzmann equation, its simplification to the BGK collision operator, the discretization of velocity space onto a lattice, and the final algorithm. By chapter's end, you will understand not just what LBM computes, but why the method works — and how to tune it for cardiac electrophysiology.

17.1 The Thermodynamic Foundation: Phase Space and the Equilibrium State

Intuition

Core Insight: Every numerical method we have studied so far — FDM, FEM, FVM — starts from a macroscopic PDE and discretizes it. LBM takes a radically different route. It starts from a much deeper level: the statistical mechanics of particles. If you correctly model how particles distribute themselves in velocity space and how that distribution evolves, the macroscopic behavior — diffusion, fluid flow, heat conduction — emerges automatically. To understand LBM, we must first understand the space it lives in and the equilibrium it relaxes toward.

Position Space: What You Already Know

In every method we have encountered so far, the unknown is a scalar field: $V(\mathbf{x}, t)$ for voltage, $T(\mathbf{x}, t)$ for temperature, $\phi(\mathbf{x}, t)$ for a generic concentration. At each point $\mathbf{x}$ in the domain, we store **one number** — the

local value of the field. This is position space: a map from location to field value. Position space is intuitive. You can picture a heat map of the heart where each pixel has a color representing voltage. FDM puts a number on each grid point. FEM interpolates between nodes. FVM averages over cells. In all cases, the fundamental object is simple: one value per spatial location. But this simplicity comes at a cost. The macroscopic PDE — the diffusion equation, the monodomain equation — is a coarse-grained description. It tells you how the *average* behavior evolves, but it has already thrown away information about what individual particles are doing. To build LBM, we need to recover some of that microscopic information. That means working in a richer space.

Velocity Space: What Are the Particles Doing?

Imagine standing at a single point in cardiac tissue and watching every ion that passes through. Some ions are moving fast to the right. Some are drifting slowly upward. Some are nearly stationary, vibrating in place. If you could catalog every ion and record its velocity, you would not get a single number — you would get a **distribution**. This distribution is the velocity-space view at a single point. It answers the question: "Of all the particles at position $\mathbf{x}$, how many have velocity $\mathbf{c}$?" In a hot gas, the distribution is wide — particles are moving at all speeds in all directions. In a cold gas, the distribution is narrow — most particles are clustered near zero velocity. In a gas with bulk flow, the distribution is shifted — its peak is no longer at zero but at the flow velocity $\mathbf{u}$. The crucial point: **at every single location in physical space, there is an entire distribution of velocities.** Not one velocity, not an average velocity — a full spread. The shape of this spread encodes the thermodynamic state at that point: temperature (width of the spread), bulk flow (shift of the peak), pressure (area under the curve).

Phase Space: Position and Velocity Together

Phase space is the combined space where you track both where a particle is AND how fast it is going. In two spatial dimensions, a particle's full state is four numbers: (x, y, c_x, c_y) — two for position, two for velocity. In three dimensions, it is six numbers: (x, y, z, c_x, c_y, c_z). The **distribution function** $f(\mathbf{x}, \mathbf{c}, t)$ lives in this combined space. It is a function of position, velocity, and time simultaneously. Its value tells you: at position $\mathbf{x}$, at time t, the density of particles with velocity $\mathbf{c}$.

$$f : (\mathbf{x}, \mathbf{c}, t) \longrightarrow \text{particle density at } (\mathbf{x}, \mathbf{c}) \text{ at time } t \tag{17.1}$$

This is the fundamental object of kinetic theory. Compare it to the macroscopic field:

	Macroscopic (FDM/FEM)	Kinetic (LBM)
Unknown	$\phi(\mathbf{x}, t)$ — one number per point	$f(\mathbf{x}, \mathbf{c}, t)$ — a distribution per point
Lives in	Position space (D dimensions)	Phase space ($2D$ dimensions)

Information	Average field value	Full velocity distribution
Storage per node	1 value	Q values (one per velocity direction)

The extra information in the velocity distribution is not wasted. It is exactly what allows LBM to avoid assembling and solving matrix systems — the transport physics is encoded in the distribution itself, and the collision-streaming algorithm updates it locally.

The Equilibrium State: Where Does the Distribution Settle?

Here is the central question of this section — and arguably the most important idea in all of LBM: **When a system of particles reaches thermal equilibrium, what shape does the velocity distribution take?** The answer has been known since the 19th century, and it is one of the great results of statistical mechanics. James Clerk Maxwell and Ludwig Boltzmann proved that the equilibrium velocity distribution of a gas is a **Gaussian** (a bell curve) in velocity space:

$$f^{\mathrm{eq}}(\mathbf{c}) = \frac{\rho}{(2\pi c_s^2)^{D/2}} \exp\left(-\frac{|\mathbf{c}-\mathbf{u}|^2}{2c_s^2}\right) \tag{17.2}$$

This is the **Maxwell-Boltzmann distribution**. Every symbol has a physical meaning:

- ρ — the total density of particles at this location. This is the macroscopic quantity we are ultimately tracking (voltage, temperature, concentration).
- $\mathbf{u}$ — the bulk (average) velocity of the particles. For diffusion problems, $\mathbf{u} = \mathbf{0}$ — there is no net flow, and the bell curve is centered at the origin.
- c_s — a parameter that controls the **width** of the bell curve. Physically, it is related to temperature: a higher c_s means a wider spread (hotter gas, faster particles on average). In LBM, c_s will become the **lattice speed of sound**, connecting the microscopic model to macroscopic transport properties.
- D — the spatial dimension (2 for 2D, 3 for 3D). The normalization $(2\pi c_s^2)^{D/2}$ ensures the total probability integrates to ρ.

What the Gaussian Tells Us: Three Physical Claims

The Maxwell-Boltzmann distribution (17.2) is not just a formula — it makes three specific physical claims about what equilibrium looks like. Each claim corresponds to a different "order" of the distribution, and each will turn out to be critically important when we build the lattice method.

Zeroth Order — Conservation

Integrate the distribution over all velocities:

$$\int f^{\text{eq}}(\mathbf{c})\, d\mathbf{c} = \rho \tag{17.3}$$

The total amount of "stuff" is ρ. This is conservation of mass (or charge, or energy, depending on what ρ represents). The equilibrium does not create or destroy anything — it only redistributes velocities. No matter how the particles rearrange their speeds and directions, the total count stays the same.

First Order — Symmetry of Flux

Compute the average velocity by weighting with $\mathbf{c}$:

$$\int \mathbf{c}\, f^{\text{eq}}(\mathbf{c})\, d\mathbf{c} = \rho\, \mathbf{u} \tag{17.4}$$

For a gas at rest ($\mathbf{u} = \mathbf{0}$), the mean velocity is zero. The bell curve is symmetric — as many particles move left as right, up as down. There is no net flux. For diffusion problems in cardiac EP, this is exactly the equilibrium condition: at rest, the distribution is centered and symmetric.

Second Order — Isotropic Variance

Compute the velocity covariance (the "spread" of the bell curve):

$$\int (c_\alpha - u_\alpha)(c_\beta - u_\beta)\, f^{\text{eq}}(\mathbf{c})\, d\mathbf{c} = \rho\, c_s^2\, \delta_{\alpha\beta} \tag{17.5}$$

The variance is **isotropic** — the bell curve has the same width in every direction. The $\delta_{\alpha\beta}$ (Kronecker delta: 1 if $\alpha = \beta$, 0 otherwise) means there is no preferred axis. The parameter c_s^2 is the variance of the Gaussian, and it is the same along x, y, and z. This isotropy is a physical claim: at thermodynamic equilibrium, no direction is special. The random thermal motion of particles is equally vigorous in all directions. This will later impose constraints on the lattice weights and will be the reason why the **standard LBM lattice produces isotropic diffusion by default**.

Intuition

Three orders, three physics: The Gaussian equilibrium encodes conservation (zeroth order), zero net flux (first order), and isotropic variance (second order). These are not abstract mathematical properties — they are the defining characteristics of thermodynamic equilibrium. Every lattice-Boltzmann scheme must respect all three, because they are the physical laws that the scheme must recover at the macroscopic level.

In Section 17.4, we will see that these three conditions are exactly the constraints that determine the lattice weights w_i and velocities $\mathbf{c}_i$.

Two Cases: Rest and Flow

The Maxwell-Boltzmann distribution (17.2) handles two physically distinct situations: **Case 1: System at rest ($\mathbf{u} = \mathbf{0}$).** The bell curve is centered at the origin of velocity space. No bulk motion — particles jiggle randomly in all directions with equal probability. This is the relevant case for **diffusion**: cardiac tissue conducts electrical signals by ion diffusion, not bulk fluid flow. The equilibrium distribution simplifies to:

$$f^{\text{eq}}(\mathbf{c}) = \frac{\rho}{(2\pi c_s^2)^{D/2}} \exp\left(-\frac{|\mathbf{c}|^2}{2c_s^2}\right) \tag{17.6}$$

a symmetric Gaussian centered at the origin. This is the equilibrium that our cardiac LBM will use. **Case 2: System with bulk flow ($\mathbf{u} \neq \mathbf{0}$).** The bell curve is shifted — its peak is at $\mathbf{u}$ instead of at the origin. More particles are moving in the flow direction than against it. This is the relevant case for **fluid dynamics** (Navier-Stokes). The distribution is still a Gaussian, but the center has moved. The shift breaks the first-order symmetry: $\int \mathbf{c}\, f^{\text{eq}}\, d\mathbf{c} = \rho\mathbf{u} \neq \mathbf{0}$.

Mathematical Note

Why c_s is the bridge between micro and macro: The parameter c_s plays a dual role. In the continuous Maxwell-Boltzmann distribution, it is the **thermal velocity** — the standard deviation of the velocity Gaussian, set by the temperature of the gas ($c_s^2 = k_B T/m$). When we discretize onto a lattice, c_s becomes the **lattice speed of sound** ($c_s^2 = c^2/3$ where $c = \Delta x/\Delta t$). This is not a coincidence: the Chapman-Enskog analysis (Section 17.2) shows that the lattice speed of sound must equal the Gaussian's variance parameter for the discrete scheme to recover the correct macroscopic transport equations. The connection $c_s^2 = c^2/3$ is a fixed consequence of the lattice geometry and the quadrature exactness conditions (Section 17.4).

Why the Equilibrium Is the Heart of LBM

In FDM, FEM, and FVM, the physics enters through the PDE — you discretize the diffusion equation or the monodomain equation directly. In LBM, **the physics enters through the equilibrium distribution**. The entire method can be summarized in one sentence: *Relax the velocity distribution toward the Maxwell-Boltzmann equilibrium, and the correct macroscopic PDE emerges automatically.* This is why we have spent this section building the equilibrium from physical principles rather than just writing down a formula. The Gaussian is not an arbitrary choice — it is the unique distribution that satisfies the constraints of thermodynamic equilibrium (maximum entropy subject to conservation of mass, momentum, and energy).

Every property of LBM — the lattice weights, the velocity sets, the collision operator, the transport coefficients — traces back to this Gaussian. In the sections that follow, we will:

- **Section 17.2:** Write down the equation that governs how f evolves toward this equilibrium (the Boltzmann equation), and simplify it for practical computation (the BGK collision operator).
- **Section 17.3:** Discretize velocity space onto a lattice, turning the continuous $f(\mathbf{x}, \mathbf{c}, t)$ into a finite set of distributions $f_i(\mathbf{x}, t)$, and write the lattice-Boltzmann equation.
- **Section 17.4:** Return to the Gaussian equilibrium and ask: what constraints do equations (17.3)–(17.5) impose on the discrete lattice? This will determine the weights w_i, velocity vectors $\mathbf{c}_i$, and the discrete equilibrium f_i^{eq}.
- **Section 17.5:** Introduce moment space — a representation that reveals which physical laws the collision operator is actually enforcing, and how to control them independently.

17.2 The Boltzmann Equation and Kinetic Theory

The Boltzmann Equation: Fundamental Form

The Boltzmann equation describes how the distribution function $f(\mathbf{x}, \mathbf{c}, t)$ — introduced in Section 17.1 — evolves in time. In the **Eulerian frame** (you stand at a fixed point in phase space and watch the distribution change around you), it reads:

$$\frac{\partial f}{\partial t} + \mathbf{c} \cdot \nabla f + \mathbf{F} \cdot \nabla_{\mathbf{c}} f = \Omega(f) \tag{17.7}$$

Let us parse each term on the left-hand side:

- $\frac{\partial f}{\partial t}$: How the distribution changes in time at a fixed point in phase space.
- $\mathbf{c} \cdot \nabla f$: Particles streaming through physical space at velocity $\mathbf{c}$. This is advection: a particle at $\mathbf{x}$ with velocity $\mathbf{c}$ will be at $\mathbf{x} + \mathbf{c}\Delta t$ after time Δt.
- $\mathbf{F} \cdot \nabla_{\mathbf{c}} f$: External forces accelerating particles, changing their velocities. The $\nabla_{\mathbf{c}}$ means the derivative is in *velocity* space, not physical space. This term accounts for gravity, electric fields, or any body force that shifts the velocity distribution.

The first two terms together form the **material (total) derivative** familiar from fluid mechanics — the rate of change following a particle that streams at velocity $\mathbf{c}$. The third term is separate: it represents forces that *change* the particle's velocity, pushing it through velocity space. If we define the streaming derivative as

$$\frac{Df}{Dt} \equiv \frac{\partial f}{\partial t} + \mathbf{c} \cdot \nabla f$$

then equation (17.7) can be written compactly as

$$\frac{Df}{Dt} + \mathbf{F} \cdot \nabla_{\mathbf{c}} f = \Omega(f) \tag{17.7a}$$

This **Lagrangian perspective** is often more intuitive: ride along with a particle that streams at constant velocity $\mathbf{c}$. From its frame, the distribution f changes for only two reasons — external forces push particles to different velocities, and collisions reshuffle the velocity population. If there were no forces and no collisions, f would be carried unchanged along the streaming trajectory, like a ball on a frictionless surface. For LBM applications (diffusion, monodomain), we will drop the external force term entirely (no body forces accelerate particles through velocity space). This makes the Eulerian form (17.7) the natural starting point for deriving the lattice-Boltzmann equation. **The right-hand side is the collision operator:** The symbol $\Omega(f)$ represents collisions. When particles collide, their velocities change. The full collision integral is:

$$\Omega(f) = \int\int (f' f_1' - f f_1)\, g\, B(g, \theta)\, d\Omega\, d\mathbf{c}_1 \tag{17.8}$$

The notation is intimidating. Here is what it means:

- f and f_1: the distribution functions of two particles BEFORE collision.
- f' and f_1': the distributions of those same two particles AFTER collision.
- $g = |\mathbf{c} - \mathbf{c}_1|$: the relative speed of the two particles.
- $B(g, \theta)$: the scattering cross-section, which depends on the relative velocity and scattering angle θ.
- $d\Omega$: integration over all possible scattering angles.
- $d\mathbf{c}_1$: integration over all velocities of the second particle.

The expression $(f' f_1' - f f_1)$ is a balance: the gain term $f' f_1'$ counts particles scattered INTO the state $(\mathbf{x}, \mathbf{c})$ by collisions, while the loss term $f f_1$ counts particles scattered OUT.

The BGK Simplification

The collision integral (17.8) is a 5-dimensional integral — computationally intractable. For a coarse 3D grid with 100^3 points and even a modest 10^3 velocities per point, evaluating it requires $\sim 10^{15}$ operations per timestep. For most of the 20th century, the Boltzmann equation remained a theoretical tool, not a practical

one. The breakthrough came from Bhatnagar, Gross, and Krook in 1954. They asked: **Can we replace the complex collision integral with something simple that still produces the correct macroscopic physics?** Their answer was to replace the collision integral with a single relaxation toward the equilibrium distribution we met in Section 17.1:

$$\Omega(f) \approx -\frac{1}{\tau}(f - f^{\text{eq}}) \tag{17.11}$$

where τ is a single relaxation time and f^{eq} is the Maxwell-Boltzmann equilibrium (equation 17.2). The idea is elegantly simple: the actual distribution f is nudged toward its local equilibrium at a rate determined by τ. No detailed calculation of scattering angles or cross-sections is needed. This is the **BGK approximation**. Notice the direct connection to Section 17.1: the f^{eq} that appears here is exactly the Gaussian equilibrium (17.2) — the bell curve in velocity space that encodes conservation, zero flux, and isotropic variance. The BGK operator says: whatever the current distribution looks like, push it toward that Gaussian. The three physical claims we established in Section 17.1 — equations (17.3), (17.4), and (17.5) — are the properties that f^{eq} carries into the collision step. Substituting (17.11) into (17.7), we get the **BGK Boltzmann equation**:

$$\frac{\partial f}{\partial t} + \mathbf{c} \cdot \nabla f = -\frac{1}{\tau}(f - f^{\text{eq}}) \tag{17.12}$$

where we have dropped the external force term $\mathbf{F} \cdot \nabla_{\mathbf{c}} f$ (in LBM for diffusion, there are no external forces accelerating particles through velocity space).

Why BGK Works: Chapman-Enskog Analysis

Why should such a drastic simplification — replacing a 5-dimensional integral with a simple relaxation — give the right macroscopic physics? The answer comes from **Chapman-Enskog analysis**, one of the deepest results in statistical mechanics. The idea: assume the distribution function is close to equilibrium, with small corrections:

$$f = f^{\text{eq}} + \epsilon f^{(1)} + \epsilon^2 f^{(2)} + \cdots$$

where ϵ is a small parameter measuring how far the system is from equilibrium (technically, the Knudsen number $\mathrm{Kn} = \lambda/L$, the ratio of mean free path to system size). Then expand the Boltzmann equation order by order. What emerges at each order is a specific piece of the macroscopic equations: **Zeroth order** (ϵ^0): the distribution equals the equilibrium $f = f^{\text{eq}}$. Taking the zeroth and first moments of f^{eq} (exactly the integrals from equations 17.3 and 17.4) gives the **conservation laws**:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0 \qquad \text{(mass conservation)}$$

These are the Euler equations — ideal fluid dynamics with no dissipation. **First order** (ϵ^1): the correction $f^{(1)}$ introduces gradients of velocity and temperature. Taking moments of $f^{(1)}$ produces the **dissipative terms** — this is where viscosity and diffusivity enter:

$$\rho \frac{D\mathbf{u}}{Dt} = -\nabla p + \underbrace{\eta \nabla^2 \mathbf{u}}_{\text{viscous stress}} + \mathbf{f} \tag{17.9}$$

for fluid flow (Navier-Stokes), and

$$\frac{\partial T}{\partial t} + \mathbf{u} \cdot \nabla T = \underbrace{D \nabla^2 T}_{\text{diffusion}} \tag{17.10}$$

for heat or mass diffusion. The underbraced terms are the ones that the first-order correction adds — they are absent in the Euler (zeroth-order) equations. The transport coefficients (viscosity η, diffusivity D) are determined by the relaxation time τ. Large τ (slow relaxation toward equilibrium) gives high viscosity or diffusivity; small τ gives low viscosity or diffusivity. The relationship is precise and will be derived explicitly in Section 17.4 as the τ-D relationship.

Intuition

The key insight: The macroscopic PDEs we have been solving throughout this textbook — the diffusion equation, the monodomain equation, even Navier-Stokes — are not fundamental. They are **emergent**. They arise as the zeroth- and first-order limits of the Boltzmann equation in the collision-dominated regime. The zeroth-order (equilibrium) contribution gives conservation laws. The first-order (near-equilibrium) correction gives the dissipative transport terms. The equilibrium f^{eq} carries the conservation physics (17.3)–(17.5); the relaxation rate $1/\tau$ sets the transport coefficients. Together, they recover the full macroscopic PDE.

Mathematical Note

Why Chapman-Enskog Works: The expansion assumes the distribution is locally near equilibrium, with small non-hydrodynamic corrections. The collision operator acts to restore equilibrium. By expanding around this equilibrium and using the fact that the collision operator has specific zero eigenvalues

(corresponding to conserved quantities like mass and momentum), one can systematically extract the macroscopic equations order by order in the Knudsen number.

17.3 The Lattice-Boltzmann Equation: Single Relaxation Time (SRT/BGK)

The Core Strategy of LBM

1. **Start from the kinetic level:** the Boltzmann equation governs how $f(\mathbf{x}, \mathbf{c}, t)$ evolves in phase space (Section 17.2).
2. **Simplify the collision:** replace the intractable collision integral with BGK relaxation toward f^{eq} — the Maxwell-Boltzmann Gaussian from Section 17.1 (Section 17.2).
3. **Discretize velocity space:** replace the continuous $\mathbf{c}$ with a small finite set of lattice velocities $\mathbf{c}_i$ and write the discrete update rule (this section).
4. **Derive the lattice from the Gaussian:** the weights w_i, velocity sets $\mathbf{c}_i$, and discrete equilibrium f_i^{eq} all follow from quadrature on the Maxwell-Boltzmann distribution (Section 17.4).
5. **Trust Chapman-Enskog:** this analysis guarantees that the simplified, discretized scheme recovers the correct macroscopic PDE in the hydrodynamic limit.

The equilibrium f^{eq} is the thread that connects everything: it defines the target of relaxation (BGK), it carries the conservation laws (Section 17.1), and it determines which macroscopic equation the scheme recovers (Chapman-Enskog).

From Continuous to Discrete: Three Deliberate Choices

In Section 17.2 we arrived at the BGK Boltzmann equation (17.12):

$$\frac{\partial f}{\partial t} + \mathbf{c} \cdot \nabla f = -\frac{1}{\tau}(f - f^{\text{eq}})$$

This is still a continuous equation — f lives on a continuous position space, a continuous velocity space, and continuous time. To turn it into something a computer can execute, we make three deliberate discretization choices: **1. Discretize velocity space.** Replace the continuous velocity $\mathbf{c}$ with a small finite set of lattice velocities $\{\mathbf{c}_0, \mathbf{c}_1, \ldots, \mathbf{c}_{Q-1}\}$. The continuous distribution $f(\mathbf{x}, \mathbf{c}, t)$ becomes a finite collection of distributions $f_i(\mathbf{x}, t)$, one for each lattice direction i. This is the step where velocity space — the infinite-dimensional space from Section 17.1 — collapses to a handful of directions. The choice of which velocities to keep (and the associated weights) is governed by the Gaussian quadrature constraints we will derive in Section 17.4. **2. Discretize time.** Replace $\partial f / \partial t$ with a finite difference over a timestep Δt. This turns the

differential equation into an update rule. **3. Lock the grid to the velocities.** Choose the spatial grid spacing Δx and timestep Δt so that each lattice velocity $\mathbf{c}_i$ moves a particle from one grid node to an adjacent grid node in exactly one timestep: $\mathbf{c}_i \Delta t = \mathbf{c}_i \Delta x / |\mathbf{c}_i|$. This eliminates interpolation entirely — streaming is exact. Notice one term that is conspicuously absent: the external force $\mathbf{F} \cdot \nabla_{\mathbf{c}} f$ from equation (17.7). Recall that this term describes forces that accelerate particles through velocity space — gravity pulling molecules downward, electric fields pushing ions. For diffusion and the monodomain equation, there are no such body forces: particles stream freely and collide, but nothing pushes them to change velocity mid-flight. We therefore drop it. (For applications that do require forcing — buoyancy-driven flows, electrokinetics — the force re-enters as a source term added to the right-hand side, but that is beyond our scope.) With these three choices, the material derivative Df/Dt becomes a simple finite difference along the lattice direction i, and the BGK equation becomes:

The Discrete Lattice-Boltzmann Equation

$$f_i(\mathbf{x},\, t + \Delta t) - f_i(\mathbf{x}, t) = -\frac{\Delta t}{\tau}\left[f_i(\mathbf{x}, t) - f_i^{\mathrm{eq}}(\mathbf{x}, t)\right] \tag{17.13}$$

The left-hand side is the change in the distribution f_i at node $\mathbf{x}$ over one timestep. The right-hand side is the collision: a relaxation nudge toward the local equilibrium f_i^{eq}, scaled by $\Delta t/\tau$. This is the entire algorithm at a single node before streaming. After computing the post-collision state, we stream: each f_i is moved to the neighboring node in the direction $\mathbf{c}_i$. The combined collision-then-stream update gives the full lattice-Boltzmann equation in its standard form:

$$f_i(\mathbf{x} + \mathbf{c}_i \Delta t,\, t + \Delta t) = f_i(\mathbf{x}, t) - \frac{\Delta t}{\tau}\left[f_i(\mathbf{x}, t) - f_i^{\mathrm{eq}}(\mathbf{x}, t)\right]$$

This single line says: compute the collision at $\mathbf{x}$, then place the result at the downstream neighbor $\mathbf{x} + \mathbf{c}_i \Delta t$. The equation is fully explicit — no matrix inversions, no implicit coupling between nodes, no iterative solves. You compute it one node at a time, and the entire simulation marches forward. In practice, we split this into two conceptual steps, which we now describe.

Step 1: Collision (Local, Pure Relaxation)

At each node, we compute:

$$f_i^*(\mathbf{x}, t) = f_i(\mathbf{x}, t) - \frac{\Delta t}{\tau}\left[f_i(\mathbf{x}, t) - f_i^{\mathrm{eq}}(\mathbf{x}, t)\right] \tag{17.14}$$

This is the collision step. The superscript $*$ on f_i^* denotes the **post-collision** distribution — the state of the distribution function AFTER the collision relaxation has been applied but BEFORE streaming moves data to neighbors. You can read f_i^* as "f-i-star" or "f-i post-collision." It is an intermediate quantity: f_i is the pre-collision state, f_i^* is the post-collision state, and after streaming, f_i^* becomes the new f_i at the destination node. The collision step happens at a single node, purely local. The distributions f_i at node $\mathbf{x}$ are nudged toward their equilibrium values f_i^{eq}. The speed of this nudge is controlled by $\Delta t/\tau$:

- If τ is large, the nudge is small — the distribution relaxes slowly.
- If τ is small (but must remain $> \Delta t/2$ for stability), the nudge is large — the distribution relaxes quickly.

The parameter τ directly controls the transport properties (diffusivity, viscosity). This is the heart of the method: all the physics (how fast heat diffuses, how viscous the fluid is) is encoded in τ.

Step 2: Streaming (Pure Data Movement)

After collision, we shift the post-collision distributions to their neighboring nodes:

$$f_i(\mathbf{x} + \mathbf{c}_i \Delta t,\, t + \Delta t) = f_i^*(\mathbf{x}, t) \tag{17.15}$$

This is the streaming step. Each distribution f_i^* at node $\mathbf{x}$ slides to the neighboring node in direction i. The destination is $\mathbf{x} + \mathbf{c}_i \Delta t$. If the lattice is designed so that $\mathbf{c}_i \Delta t = \Delta x$ (one lattice spacing), then streaming is a simple data copy — a value at the center moves to a neighbor. Critically, streaming is not a collision. There is no physics in streaming — it is pure kinematics. A particle moving with velocity $\mathbf{c}_i$ will naturally travel a distance $\mathbf{c}_i \Delta t$ in time Δt. Streaming enforces this.

The Combined Form

Substituting the collision step (17.14) into the streaming step (17.15) — replacing f_i^* with its definition — gives the single combined equation:

$$f_i(\mathbf{x} + \mathbf{c}_i \Delta t,\, t + \Delta t) = f_i(\mathbf{x}, t) - \frac{\Delta t}{\tau}\left[f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\mathbf{x}, t)\right] \tag{17.16}$$

This is the **lattice-Boltzmann equation in streaming form**. It is the same physics as equation (17.13), but written from the destination's perspective: the value arriving at node $\mathbf{x} + \mathbf{c}_i \Delta t$ at the next timestep equals the current value at node $\mathbf{x}$, corrected by the collision. You will encounter this form throughout the LBM literature — it is the most compact way to write the full collide-and-stream update.

Intuition

Locality and Parallelization: Both collision and streaming are local operations. Collision affects only one node at a time. Streaming is a data copy — the value $f_i^*(\mathbf{x}, t)$ moves to a neighbor, but it does not interact with neighbors' data. After collision, all the f_i at all nodes are independent. Streaming then redistributes them. This extreme locality means LBM is trivially parallelizable: each node (or even each lattice direction at each node) can be processed on a separate processor with zero communication overhead during the step. Only a halo exchange is needed between processors after streaming. This is one reason LBM is popular on GPUs.

Variable Definitions and Component Form

To implement LBM, you must understand every variable. Here is a complete reference:

Symbol	Meaning	Units / Constraints
$f_i(\mathbf{x}, t)$	Distribution function for direction i at node $\mathbf{x}$, time t	Density (or concentration if diffusion)
$f_i^*(\mathbf{x}, t)$	Post-collision distribution for direction i (after relaxation, before streaming)	Same units as f_i
$f_i^{\text{eq}}(\mathbf{x}, t)$	Equilibrium distribution for direction i	Density (or concentration)
τ	Relaxation time	Must satisfy $\tau > \Delta t/2$
Δt	Timestep	In lattice units, $\Delta t = 1$
$\mathbf{c}_i$	Discrete velocity vector for direction i	$\mathbf{c}_0 = \mathbf{0}$ (rest); $\lvert\mathbf{c}_i\rvert = c = \Delta x/\Delta t$ (moving)
w_i	Weight for direction i	$\sum_i w_i = 1$
$\phi(\mathbf{x}, t) = \sum_i f_i$	Macroscopic variable (temperature, voltage, density)	Physical units (K, mV, kg/m³)
Q	Number of lattice directions (velocities)	i ranges from 0 to $Q - 1$

In detail: $f_i(\mathbf{x}, t)$ is the distribution function for direction i. Think of it as "the portion of the macroscopic quantity assigned to direction i at this node at this time." There are Q values of f_i stored at each node — one per lattice direction. The sum $\sum_i f_i = \phi$ recovers the total macroscopic quantity. $f_i^{\text{eq}}(\mathbf{x}, t)$ is the equilibrium distribution for direction i — the target that the collision step relaxes toward. Its exact form depends on the weights w_i and the macroscopic field, and will be derived in Section 17.4 from the Gaussian equilibrium we

built in Section 17.1. τ is the relaxation time. It controls how quickly the distribution approaches equilibrium. The relationship between τ and the physical diffusivity D comes from Chapman-Enskog analysis and will be derived in Section 17.4. Δt is the timestep. In lattice units, we often set $\Delta t = 1$, but it is useful to keep Δt explicit when connecting lattice variables to physical ones. $\mathbf{c}_i$ is the discrete velocity vector for direction i. Each $\mathbf{c}_i$ points from a node to a neighbor (or stays put if $i = 0$). The specific velocity sets and their associated weights are derived in Section 17.4 from the Gaussian quadrature framework. w_i is the weight for direction i. Like $\mathbf{c}_i$, the weights are uniquely determined by the quadrature exactness conditions and will be derived in Section 17.4. $\phi(\mathbf{x}, t) = \sum_i f_i(\mathbf{x}, t)$ is the recovered macroscopic quantity. For diffusion of temperature, ϕ is temperature. For cardiac electrophysiology, ϕ is the voltage. The sum over all lattice directions gives the total.

17.4 From the Gaussian to the Lattice: Discretizing f^{eq}, w_i, and $\mathbf{c}_i$

Returning to the Gaussian

In Section 17.1 we established the Maxwell-Boltzmann equilibrium (equation 17.2) and its three physical claims: conservation of mass (17.3), zero net flux at rest (17.4), and isotropic variance (17.5). In Section 17.2, this equilibrium became the target of BGK relaxation — the distribution f is nudged toward f^{eq} at every collision. In Section 17.3, we wrote the discrete lattice-Boltzmann equation and used f_i^{eq}, w_i, and $\mathbf{c}_i$ as if they were known quantities — but we never said where they come from. Now we open that box. Everything in this section — the discrete velocities, the weights, and the discrete equilibrium — is derived from a single source: the continuous Gaussian from Section 17.1. The question is: **how do you discretize a bell curve in velocity space so that the discrete version preserves the same physical claims?**

The Three Claims Become Three Constraints on the Lattice

Recall the three physical claims of the Gaussian equilibrium from Section 17.1. Each was stated as a continuous integral over velocity space. When we discretize velocity space onto a finite set of Q directions $\{\mathbf{c}_i\}$ with weights $\{w_i\}$, each integral becomes a finite sum — and each physical claim becomes an algebraic constraint that the lattice must satisfy. **Zeroth order — conservation of mass** (discrete form of equation 17.3):

$$\sum_i w_i = 1 \qquad \text{(normalization: the total probability is 1)} \tag{17.18}$$

The continuous claim $\int f^{\text{eq}} d\mathbf{c} = \rho$ says the total density is conserved. In discrete form, this requires the weights to sum to one — the discrete bell curve must have the same total area as the continuous one. **First order — zero net flux at rest** (discrete form of equation 17.4):

$$\sum_i w_i\, c_{i\alpha} = 0 \qquad \text{or equivalently:} \qquad \sum_i w_i\, \mathbf{c}_i = \mathbf{0} \tag{17.19}$$

The continuous claim $\int \mathbf{c}\, f^{\text{eq}} d\mathbf{c} = \rho \mathbf{u} = \mathbf{0}$ (at rest) says there is no net momentum. In discrete form, the weighted average of the velocity vectors must vanish — the lattice must be symmetric so that no direction is preferred. **Second order — isotropic variance** (discrete form of equation 17.5):

$$\sum_i w_i\, c_{i\alpha}\, c_{i\beta} = c_s^2\, \delta_{\alpha\beta} \qquad \text{or equivalently:} \qquad \sum_i w_i\, \mathbf{c}_i \otimes \mathbf{c}_i = c_s^2\, \mathbf{I} \tag{17.20}$$

The continuous claim $\int c_\alpha c_\beta f^{\text{eq}} d\mathbf{c} = \rho c_s^2 \delta_{\alpha\beta}$ says the velocity fluctuations are isotropic. In discrete form, the weighted outer product of velocities must reproduce the variance of the Gaussian times the identity matrix. When $\alpha = \beta$ (e.g., $\alpha = \beta = x$), this says $\sum_i w_i c_{ix}^2 = c_s^2$ — the quadrature reproduces the Gaussian's variance along each axis. When $\alpha \neq \beta$ (e.g., $\alpha = x$, $\beta = y$), this says $\sum_i w_i c_{ix} c_{iy} = 0$ — the x and y components are uncorrelated, as required for an isotropic bell curve. These three constraints — (17.18), (17.19), and (17.20) — are the bridge between the continuous physics of Section 17.1 and the discrete lattice. Any valid lattice (D2Q5, D2Q9, D3Q7) must satisfy all three. The constraints determine the weights w_i once the velocity set $\mathbf{c}_i$ is chosen.

Gauss-Hermite Quadrature: Why These Specific Nodes and Weights

The three constraints above are not arbitrary — they are the **exactness conditions of Gauss-Hermite quadrature**. This is a classical result in numerical analysis: given a function that looks like a polynomial multiplied by a Gaussian, how do you approximate the integral using a finite sum? The idea is beautifully geometric. The Gaussian bell curve is a hill in velocity space, and we want to compute integrals under it. Gauss-Hermite quadrature tells us exactly where to place sample points (the **nodes** $\mathbf{c}_i$) and how much weight to give each one (the **weights** w_i), so that the finite sum is exact for all polynomials up to a certain degree:

$$\int p(\mathbf{c})\, \underbrace{\frac{1}{(2\pi c_s^2)^{D/2}} e^{-|\mathbf{c}|^2/2c_s^2}}_{\text{Gaussian kernel}}\, d\mathbf{c} \;\approx\; \sum_{i=0}^{Q-1} w_i\, p(\mathbf{c}_i) \tag{17.21}$$

The quadrature nodes become our discrete velocities. The quadrature weights become our lattice weights. The constraints (17.18)–(17.20) are what you get when you require this sum to be exact for $p = 1$ (zeroth order), $p = c_\alpha$ (first order), and $p = c_\alpha c_\beta$ (second order) — precisely the three physical claims from Section 17.1.

Intuition

Nodes and Weights Are a Married Pair: In Gauss-Hermite quadrature, you do not get to choose the nodes and the weights independently. Once you pick a set of sample points on the bell curve, the weights are uniquely determined (and vice versa). This is why every lattice specification — D2Q5, D2Q9, D3Q7 — always lists both $\mathbf{c}_i$ AND w_i together. They are a package deal, born from the same Gaussian.

For **diffusion** (no bulk flow), the equilibrium is the Gaussian centered at zero. The macroscopic equations involve integrals of polynomials up to second degree in $\mathbf{c}$, so exactness through second order — constraints (17.18)–(17.20) — is sufficient. D2Q5 and D3Q7 satisfy these. For **fluid flow** (Navier-Stokes), the equilibrium is a shifted Gaussian, and higher-order polynomial terms appear. The quadrature must also be exact for fourth-degree polynomials, which requires additional exactness conditions and more lattice directions (D2Q9 instead of D2Q5).

Discrete vs. Continuous: Lattice Naming Convention

Lattices are named using the notation $\mathrm{D}_d\mathrm{Q}_q$, where:

- **D** = spatial dimension (1, 2, or 3)
- **Q** = number of lattice directions (discrete velocities)

Common examples:

- **D1Q3**: 1D, 3 velocities (left, right, rest)
- **D2Q5**: 2D, 5 velocities (rest + 4 cardinal neighbors)
- **D2Q9**: 2D, 9 velocities (rest + 4 cardinal + 4 diagonal)
- **D3Q7**: 3D, 7 velocities (rest + 6 face-centered neighbors)
- **D3Q19**: 3D, 19 velocities (face-centered + edge-adjacent)
- **D3Q27**: 3D, 27 velocities (complete cubic stencil)

The choice of lattice is determined by the physics:

- **For diffusion** (no bulk flow), D2Q5 or D3Q7 suffice. These have only cardinal directions.
- **For fluid flow** (Navier-Stokes), diagonal directions are needed to represent the full stress tensor. D2Q9 or D3Q19 are required.

Since cardiac electrophysiology primarily solves diffusion problems (the monodomain and bidomain equations), **D2Q5 and D3Q7 are the natural choices**.

⚠ Key Insight

Memory vs. Physics: The number of distributions stored per node equals Q. A D2Q9 lattice requires storing 9 values per node; a D2Q5 requires only 5. For a 500×500×500 3D mesh, a D2Q5 saves a factor of $(\text{7 directions})/(\text{27 directions}) \approx 4\times$ memory compared to D3Q27, even though physically, D3Q7 produces the same diffusion results. This is one reason LBM for cardiac problems uses minimal lattices.

The Discrete Velocity Set

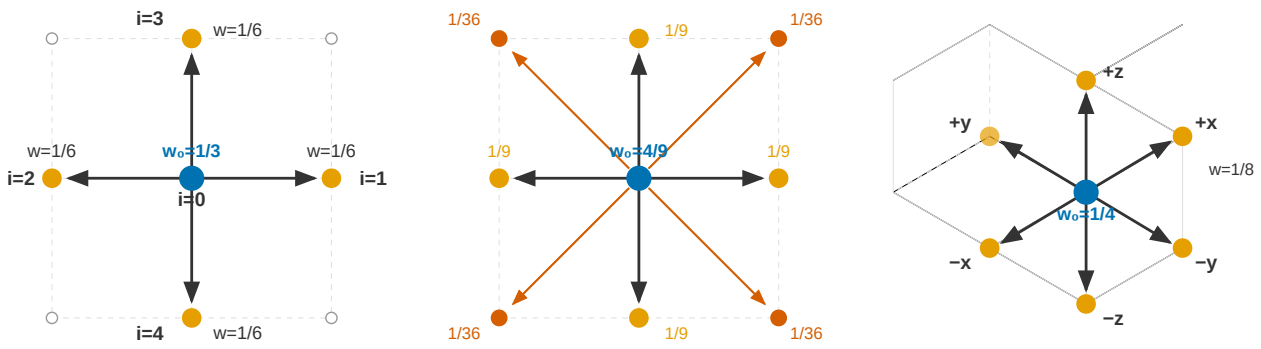
With the quadrature framework in mind, here are the standard lattices. Each is a specific Gauss-Hermite quadrature rule on a regular grid — the nodes are placed at grid points so that streaming is a simple data shift. **D1Q3 (One-Dimensional):**

$$\mathbf{c}_0 = 0, \quad \mathbf{c}_1 = c, \quad \mathbf{c}_2 = -c \tag{17.17}$$

where $c = \Delta x/\Delta t$ is the lattice speed. Three quadrature nodes: one at the center of the bell curve (rest), one to the right, one to the left. **D2Q5 (Two-Dimensional, Cardinal Directions):**

i	Direction	$\mathbf{c}_i$ (in units of c)
0	Rest	$(0, 0)$
1	East	$(1, 0)$
2	West	$(-1, 0)$
3	North	$(0, 1)$
4	South	$(0, -1)$

Five quadrature nodes: one at the peak of the 2D bell curve (rest), four at one lattice spacing along each cardinal direction. These five sample points are sufficient to exactly integrate second-degree polynomials against the 2D Gaussian.



(a) D2Q5 (b) D2Q9 (c) D3Q7

*Figure 17.1: Common lattice stencils for LBM. **(a)** D2Q5: five velocities (rest + four cardinal) — sufficient for isotropic diffusion. **(b)** D2Q9: nine velocities (rest + cardinal + diagonal) — required for fluid flow to capture cross-velocity terms. **(c)** D3Q7: seven velocities in 3D (rest + six face-centered) — the minimal 3D stencil for diffusion. Blue center node = rest velocity (peak of the Gaussian); orange = cardinal neighbors; vermillion = diagonal neighbors. Arrows indicate streaming directions; weights w_i annotated at each node.*

D2Q9 (Two-Dimensional, Cardinal + Diagonal): For fluid flow, the polynomial expansion of the shifted Gaussian contains terms like $c_{ix}c_{iy}$ (cross-products of velocity components). To integrate these exactly, we need diagonal quadrature nodes. D2Q9 adds four diagonal directions to D2Q5:

i	Direction	$\mathbf{c}_i$ (in units of c)
0	Rest	$(0, 0)$
1	East	$(1, 0)$
2	West	$(-1, 0)$
3	North	$(0, 1)$
4	South	$(0, -1)$
5	NE	$(1, 1)$
6	NW	$(-1, 1)$
7	SE	$(1, -1)$
8	SW	$(-1, -1)$

Directions 1–4 are the same cardinal nodes as D2Q5. Directions 5–8 are diagonal nodes at distance $\sqrt{2}\,c$ from the center. These sit further out on the bell curve than the cardinal nodes, sampling the Gaussian's tails in diagonal directions. **D3Q7 (Three-Dimensional, Face-Centered):** The 3D analogue of D2Q5. One quadrature node at the peak, six at one lattice spacing along each coordinate axis:

i	Direction	$\mathbf{c}_i$ (in units of c)
0	Rest	$(0, 0, 0)$
1	$+x$	$(1, 0, 0)$
2	$-x$	$(-1, 0, 0)$
3	$+y$	$(0, 1, 0)$
4	$-y$	$(0, -1, 0)$
5	$+z$	$(0, 0, 1)$
6	$-z$	$(0, 0, -1)$

All six moving nodes sit at the same distance c from the center — they sample the Gaussian at the same height on the bell curve, which is why they all receive the same weight.

The Weight Lattice

The weights paired with each velocity set are determined by the quadrature exactness conditions (17.18)–(17.20). Each weight w_i represents how much of the Gaussian bell curve the quadrature node $\mathbf{c}_i$ is responsible for. Nodes near the peak of the Gaussian (where the bell curve is tallest) represent more of the total area and receive larger weights. Nodes further out (where the bell curve has decayed) represent less area and receive smaller weights. **D1Q3 weights:**

$$w_0 = \frac{2}{3}, \quad w_1 = w_2 = \frac{1}{6} \tag{17.22}$$

The rest node sits at the peak of the 1D bell curve — it captures two-thirds of the total area. The two moving nodes, one lattice spacing away from the peak, each capture one-sixth. Most of the Gaussian's area is concentrated near the center, which is why the rest weight dominates. **D2Q5 weights:**

$$w_0 = \frac{1}{3}, \quad w_1 = w_2 = w_3 = w_4 = \frac{1}{6} \tag{17.23}$$

The rest node captures one-third of the 2D bell curve. Each cardinal node captures one-sixth. The total is $1/3 + 4 \times 1/6 = 1$. All four cardinal nodes are at the same distance from the peak, so they receive the same weight — the bell curve is symmetric. **D2Q9 weights:**

$$w_0 = \frac{4}{9}, \quad w_{1,2,3,4} = \frac{1}{9} \quad \text{(cardinal)}, \quad w_{5,6,7,8} = \frac{1}{36} \quad \text{(diagonal)} \tag{17.24}$$

Three distinct weight classes, directly reflecting the Gaussian's shape. The rest node at the peak captures $4/9$ of the area — nearly half. The cardinal nodes at distance c each capture $1/9$. The diagonal nodes at distance $\sqrt{2}\,c$ each capture only $1/36$ — four times less than a cardinal node. This factor of four is the Gaussian at work: moving from distance c to $\sqrt{2}\,c$ from the peak, the bell curve has decayed, so those nodes represent less area. Check: $4/9 + 4 \times 1/9 + 4 \times 1/36 = 4/9 + 4/9 + 1/9 = 1$ ✓.

Intuition

Why Diagonal Weights Are Smaller: Picture a 2D bell curve — a smooth hill centered at the origin. The cardinal quadrature nodes sit at distance c from the peak (along the axes). The diagonal nodes sit at distance $\sqrt{2}\,c$ (along the diagonals). Because the Gaussian decays exponentially with distance from the center, the bell curve is lower at the diagonal nodes than at the cardinal nodes. Lower height means less area to represent, which means a smaller quadrature weight. This is not a choice — it is a consequence of the Gaussian's shape. The ratio $w_{\text{diagonal}}/w_{\text{cardinal}} = (1/36)/(1/9) = 1/4$ is determined by the exactness conditions, but the physical reason is the Gaussian decay.

D3Q7 weights:

$$w_0 = \frac{1}{4}, \quad w_1 = \cdots = w_6 = \frac{1}{8} \tag{17.25}$$

The rest node captures one-quarter of the 3D bell curve. Each of the six face-centered nodes captures one-eighth. All moving nodes are at the same distance c from the peak (same height on the bell curve), so they receive equal weight. Check: $1/4 + 6 \times 1/8 = 1/4 + 3/4 = 1$ ✓.

Mathematical Note

Deriving D2Q5 Weights from Exactness: For D2Q5, we have two unknowns: the rest weight w_0 and the cardinal weight w_c (shared by directions 1–4 by symmetry). Two conditions pin them down. First, normalization (17.18): $w_0 + 4w_c = 1$. Second, the variance condition (17.20) applied to the x-direction: $\sum_i w_i c_{ix}^2 = c_s^2$. The rest particle contributes zero. Directions 1 and 2 each contribute $w_c \cdot c^2$. Directions 3 and 4 have $c_{ix} = 0$, so they contribute nothing. This gives $2w_c c^2 = c_s^2 = c^2/3$, yielding $w_c = 1/6$. Substituting back: $w_0 = 1 - 4(1/6) = 1/3$. The weights are not chosen — they are uniquely determined by the quadrature exactness conditions on this lattice geometry.

The Discrete Equilibrium f_i^{eq}: Closing the Loop

We can now close the loop that has been open since Section 17.1. There, we wrote the continuous equilibrium — the Maxwell-Boltzmann Gaussian (equation 17.2). In Section 17.3, we used f_i^{eq} as a black box in the collision step (equation 17.14). Here is what that black box actually contains. For **pure diffusion** ($\mathbf{u} = 0$), the Gaussian is centered at the origin. When you evaluate it at the quadrature nodes and fold in the weights, all the velocity-dependent structure vanishes. The discrete equilibrium collapses to:

$$f_i^{\text{eq}} = w_i\,\phi \tag{17.30}$$

where $\phi = \sum_j f_j$ is the local macroscopic quantity (temperature, voltage, concentration). Each direction carries a fraction w_i of the total field — exactly the quadrature weight. The rest direction carries the most (it sits at the peak of the bell curve), and the moving directions carry the remainder, split according to how much of the Gaussian each node represents. There is no preferred direction — the equilibrium is symmetric, just as the centered Gaussian is symmetric. This is the equilibrium we use for cardiac electrophysiology, since the monodomain and bidomain equations are diffusion equations with no bulk flow. For **fluid flow** (Navier-Stokes), the Gaussian is centered at the bulk velocity $\mathbf{u}$. Expanding the exponential in (17.2) as a Taylor series in $\mathbf{u}/c_s$ (valid when the flow speed is much smaller than the speed of sound — the low Mach number regime), the discrete equilibrium becomes:

$$f_i^{\text{eq}} = w_i \rho \left(1 + \frac{\mathbf{u} \cdot \mathbf{c}_i}{c_s^2} + \frac{(\mathbf{u} \cdot \mathbf{c}_i)^2}{2c_s^4} - \frac{u^2}{2c_s^2}\right) \tag{17.31}$$

The terms beyond $w_i \rho$ are the polynomial corrections that arise from shifting the bell curve's center from zero to $\mathbf{u}$. These terms break the directional symmetry — they bias the distribution toward the direction of flow. Because these corrections involve products like $c_{ix} c_{iy}$, the quadrature must exactly integrate fourth-degree polynomials, which is why fluid flow requires more quadrature nodes (D2Q9 instead of D2Q5).

Lattice Requirements for Different Physics

The choice of lattice is determined by how many terms in the polynomial expansion of the Gaussian need to be captured exactly. For diffusion with the isotropic equilibrium (17.30), only terms up to second degree matter — the exactness conditions (17.18)–(17.20) suffice. D2Q5 (and D3Q7 in 3D) satisfy these, making them the most efficient choices for cardiac problems. For fluid flow with the velocity-dependent equilibrium (17.31), terms up to fourth degree appear. The quadrature must satisfy additional exactness conditions involving the fourth-order product $\sum_i w_i c_{i\alpha} c_{i\beta} c_{i\gamma} c_{i\delta}$. D2Q5 does not have enough nodes to satisfy these, but D2Q9 does. In cardiac electrophysiology, we solve diffusion (not Navier-Stokes), so D2Q5 and D3Q7 are sufficient. Using a minimal lattice saves both memory (Q distributions per node) and computation (fewer collision and streaming operations per timestep).

Lattice Speed of Sound and the τ-D Relationship

For each lattice, there is a characteristic speed c_s called the **lattice speed of sound**. From the isotropy condition (17.20) and the lattice geometry, we have:

$$c_s^2 = \frac{c^2}{3} \tag{17.26}$$

The parameter c_s plays a dual role. In the continuous Gaussian of Section 17.1, c_s^2 is the **variance** — it controls the width of the bell curve. On the lattice, this same c_s becomes the **lattice speed of sound**, with the specific value $c_s^2 = c^2/3$ where $c = \Delta x/\Delta t$ is the lattice speed. This is not a coincidence: the lattice speed of sound is inherited directly from the variance of the Maxwellian that the lattice discretizes. Every time you see c_s^2 in an LBM formula — in the weights, in the equilibrium, in the τ-D relationship — it traces back to this Gaussian width. The speed of sound c_s is just a lattice constant — it emerges from the geometry and isotropy conditions. However, it plays a crucial role in relating the relaxation time τ to the physical diffusivity D. Through Chapman-Enskog analysis, one can show that for the BGK collision operator applied to a diffusion problem, the relationship is:

$$D = c_s^2 \left(\tau - \frac{\Delta t}{2}\right) \tag{17.27}$$

or, rearranging:

$$\tau = \frac{\Delta t}{2} + \frac{D}{c_s^2} \tag{17.28}$$

Substituting $c_s^2 = c^2/3 = \Delta x^2/(3\Delta t^2)$:

$$\tau = \frac{\Delta t}{2} + \frac{3D\Delta t}{\Delta x^2} \tag{17.29}$$

This is the fundamental bridge between the discrete lattice parameter τ and the physical diffusivity D. **Practical meaning:** To solve a problem with a given physical diffusivity D, you:

1. Choose your spatial and temporal resolution: Δx and Δt.
2. Compute τ from equation (17.29).
3. Use that τ in the collision step (equation 17.14).

Note the stability constraint: $\tau > \Delta t/2$ is required for stability. This places an upper bound on D for a given Δx and Δt. If you need very high diffusivity, you must either reduce Δx or accept a smaller Δt, which increases the total number of timesteps.

⚠ Key Insight

Stability Constraint: The requirement $\tau > \Delta t/2$ comes from the local truncation error analysis of the BGK collision operator. If $\tau \leq \Delta t/2$, the relaxation becomes too aggressive and unphysical oscillations grow. This is similar to the CFL condition in hyperbolic PDEs — it is a fundamental constraint on the simulation. For a fixed spatial grid, high diffusivity requires a small Δt, which can make the simulation expensive.

17.5 Moment Space: The Physical Decomposition of the Collision Operator

The Problem: What Does the Collision Operator Actually Conserve?

Look again at the BGK collision equation we derived in Section 17.3:

$$f_i^*(\mathbf{x}, t) = f_i(\mathbf{x}, t) - \frac{\Delta t}{\tau}\left[f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\mathbf{x}, t)\right] \tag{17.32}$$

This equation relaxes each distribution f_i toward its equilibrium value f_i^{eq}, all at the same rate $1/\tau$. It works — we showed in Section 17.4 that the equilibrium was carefully designed to encode the right physics. But ask yourself: what *guarantees* that this relaxation conserves mass? What guarantees it produces the correct diffusion coefficient? What guarantees the stress tensor has the right symmetry? The answers are hiding inside f^{eq}, but equation (17.32) doesn't show them. Each f_i is a single number that mixes together information about density, flux, stress, and energy — all tangled up. When BGK relaxes f_1 toward f_1^{eq}, it is simultaneously relaxing *every* physical quantity that f_1 carries, at the same rate. You can't see which conservation laws are being enforced, and you have no way to control them independently. Is there a way to **see** what physical laws the collision operator is actually enforcing — and to **verify** them directly?

The Macroscopic PDE Dictates Which Moments Matter

The answer starts not with the lattice, but with the PDE you are trying to solve. Consider the diffusion equation:

$$\frac{\partial \phi}{\partial t} = \nabla \cdot (D \nabla \phi) \tag{17.33}$$

This PDE makes three specific physical demands, and each one maps to a weighted sum of the distributions:
Demand 1 — Conservation of mass (or charge, or heat): The total amount of ϕ at a node does not change during collision. Only streaming moves ϕ between nodes. This means:

$$\sum_{i=0}^{Q-1} f_i^* = \sum_{i=0}^{Q-1} f_i \tag{17.34}$$

The sum $\sum_i f_i$ is a weighted sum of the distributions with weight 1 for every direction. This is the **zeroth moment** — the simplest possible summary of the distributions. You take it because *mass conservation demands it*. **Demand 2 — Fick's law (diffusive flux):** The macroscopic flux must satisfy $\mathbf{J} = -D\nabla\phi$. In the lattice-Boltzmann framework, the flux emerges from:

$$J_\alpha = \sum_{i=0}^{Q-1} c_{i\alpha} f_i \tag{17.35}$$

These sums weight each distribution by its velocity component $c_{i\alpha}$. They are the **first moments**. You take them because *Fick's law demands them* — the diffusion coefficient is encoded in how fast these flux quantities relax toward equilibrium. **Demand 3 — Correct tensor structure:** For the diffusion to be isotropic (or anisotropic in exactly the right way), the stress-like quantity:

$$\Pi_{\alpha\beta} = \sum_{i=0}^{Q-1} c_{i\alpha} c_{i\beta} f_i \tag{17.36}$$

must have the correct symmetry. These are the **second moments**. You take them because *the diffusion tensor structure demands them*. For isotropic diffusion, $\Pi_{\alpha\beta}$ must be proportional to $\delta_{\alpha\beta}$ at equilibrium. For anisotropic diffusion, its structure must match the physical diffusion tensor.

Intuition

The Rule: Each conservation law or transport property in your target PDE maps to a specific weighted sum of the f_i. These weighted sums *are* the moments. You don't choose them for mathematical convenience — the physics tells you which ones matter. If you are solving a diffusion equation, you need the zeroth moment (conservation), the first moments (flux), and the second moments (tensor structure). If you were solving Navier-Stokes, the same zeroth and first moments would enforce mass and momentum conservation, but the second moments would need to capture the viscous stress tensor, and you might need third- and fourth-order moments for the energy equation.

What Are Moments?

The pattern is now clear. A **moment** is a weighted sum of the distribution functions, where the weights are chosen to extract a specific macroscopic quantity. In the most general form:

$$m_k = \sum_{i=0}^{Q-1} M_{ki}\, f_i \tag{17.37}$$

where M_{ki} is the weight that the k-th moment assigns to the i-th distribution. This is not unique to LBM — it is the same concept that appears across all of engineering:

- **Zeroth moment** ($M_{0i} = 1$ for all i): total density, total charge, total energy. The conserved quantity in any transport equation.
- **First moments** ($M_{ki} = c_{i\alpha}$): momentum flux in fluids, diffusive flux in electrophysiology, heat flux in conduction. The direction and rate at which "stuff" moves.
- **Second moments** ($M_{ki} = c_{i\alpha} c_{i\beta}$): the viscous stress tensor in fluid mechanics, the diffusion tensor in EP, the energy density in heat transfer. These control how transport varies with direction.
- **Higher moments** (M_{ki} involves cubic or higher products of $c_{i\alpha}$): in most diffusion and low-Mach flow problems, no conservation law requires them. They exist because the lattice has Q degrees of freedom but only a few physical constraints. These are the **ghost moments** — mathematically necessary to fill out the Q-dimensional space, but carrying no macroscopic physical content.

Building M: Each Row Is a Conservation Law

We can collect all the moment definitions into a single matrix equation:

$$\mathbf{m} = \mathbf{M}\,\mathbf{f} \tag{17.38}$$

where $\mathbf{m} = (m_0, m_1, \ldots, m_{Q-1})^T$ is the vector of moments and $\mathbf{f} = (f_0, f_1, \ldots, f_{Q-1})^T$ is the vector of distributions. The matrix $\mathbf{M}$ is the **transformation matrix** — each row is the recipe for extracting one physical quantity from the distributions. For **D2Q5**, we have $Q = 5$ distributions and therefore need a 5×5 matrix. The rows are constructed directly from the physics:

Row	Moment	Physical meaning	Why we need it
0	$\sum_i f_i$	Density (conserved scalar ϕ)	Mass/charge conservation
1	$\sum_i c_{ix} f_i$	x-flux	Fick's law in x
2	$\sum_i c_{iy} f_i$	y-flux	Fick's law in y
3	$\sum_i \lvert\mathbf{c}_i\rvert^2 f_i$	Energy	Ghost — no PDE requires it

4	$\sum_i (c_{ix}^2 - c_{iy}^2) f_i$	Energy difference (stress trace)	Ghost — no PDE requires it

The explicit 5×5 matrix, using the D2Q5 velocity vectors (c_{ix}, c_{iy}) = $(0,0)$, $(1,0)$, $(-1,0)$, $(0,1)$, $(0,-1)$:

$$\mathbf{M}_{\text{D2Q5}} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & -1 & -1 \end{pmatrix} \tag{17.39}$$

Read it row by row. Row 0: every entry is 1, so this row sums all f_i — that's the density. Row 1: the entries are the x-components of the velocity vectors, so this row computes the x-flux. Row 2: the y-components — the y-flux. Rows 3 and 4: combinations of c_{ix}^2 and c_{iy}^2 that complete the basis. The first three rows are *demanded by the physics*. The last two rows are chosen to make $\mathbf{M}$ invertible (so we can transform back), but any linearly independent choice would work. For **D3Q7**, the 7×7 transformation matrix from the Rapaka et al. LBM-EP paper follows the same logic. With velocity ordering $(0,0,0)$, $(1,0,0)$, $(-1,0,0)$, $(0,1,0)$, $(0,-1,0)$, $(0,0,1)$, $(0,0,-1)$:

$$\mathbf{M}_{\text{D3Q7}} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & -1 \\ -6 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & -1 & -1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & -2 & -2 \end{pmatrix} \tag{17.40}$$

Rows 0–3: density and the three flux components (x, y, z) — required by mass conservation and Fick's law in 3D. Rows 4–6: ghost moments with no macroscopic counterpart, chosen to complete the 7×7 basis.

Mathematical Note

Why the ghost rows look arbitrary: The first rows of $\mathbf{M}$ are dictated by physics — row 0 must be all ones (density), rows 1–D must use the velocity components (flux). The remaining $Q - 1 - D$ rows just need to be linearly independent from the physical rows. Different textbooks choose different ghost rows, and *all valid choices produce the same physical solution*. The ghost rows affect only numerical stability, not the macroscopic PDE that the scheme recovers.

The Round Trip: Rewriting the Collision Operator in Moment Space

Now we have the tool to see inside the collision operator. The key idea is simple: instead of relaxing the f_i directly (where every physical quantity is tangled together), we **decode** the distributions into moments, **relax** each moment separately, then **re-encode** back to distributions for streaming. **Step 1 — Decode:** Transform the distributions into moments:

$$\mathbf{m} = \mathbf{M}\,\mathbf{f} \tag{17.41}$$

Now m_0 is the density, m_1 is the x-flux, m_2 is the y-flux, and so on. Each component has exactly one physical meaning. **Step 2 — Decode the equilibrium too:**

$$\mathbf{m}^{\text{eq}} = \mathbf{M}\,\mathbf{f}^{\text{eq}} \tag{17.42}$$

The equilibrium moments tell you the target state for each physical quantity. For the diffusion equilibrium $f_i^{\text{eq}} = w_i \phi$, the equilibrium density is $m_0^{\text{eq}} = \phi$ (all the field), the equilibrium fluxes are $m_1^{\text{eq}} = m_2^{\text{eq}} = 0$ (no net flow at equilibrium), and the ghost moments take whatever values w_i and the lattice geometry dictate. **Step 3 — Relax in moment space:**

$$\mathbf{m}^* = \mathbf{m} - \mathbf{S}\left(\mathbf{m} - \mathbf{m}^{\text{eq}}\right) \tag{17.43}$$

where $\mathbf{S} = \text{diag}(s_0, s_1, \ldots, s_{Q-1})$ is the **relaxation matrix**. Each diagonal entry s_k controls how fast moment k relaxes toward its equilibrium value. This is the step where the physics is enforced:

- For the conserved moment (density): $s_0 = 0$. The density is never changed by collision — it is a conserved quantity. Setting $s_0 = 0$ means $m_0^* = m_0$ exactly, regardless of what happens to the other moments.
- For the flux moments: s_1 and s_2 control how fast the flux relaxes, which determines the diffusion coefficient. In the simplest case (isotropic BGK), $s_1 = s_2 = \Delta t/\tau$.
- For the ghost moments: $s_3, s_4, \ldots$ can be chosen freely — they do not affect the macroscopic PDE, only the numerical behavior of the scheme.

Step 4 — Re-encode: Transform back to distribution space:

$$\mathbf{f}^* = \mathbf{M}^{-1}\,\mathbf{m}^* \tag{17.44}$$

Now $\mathbf{f}^*$ is ready for streaming. The distributions have been updated in a way that *explicitly* conserves density, *explicitly* relaxes flux at a controlled rate, and handles the ghost moments separately.

Intuition

The destination vs. the journey: It is worth pausing to see how f^{eq} and moment space play complementary roles. The equilibrium f_i^{eq} defines the **destination** — the state the distributions are relaxing toward. It encodes *what* the physics looks like at equilibrium (isotropic for diffusion, velocity-biased for fluid flow). Moment space governs the **journey** — *how* the distributions get to that destination and *what is preserved along the way*. By decomposing the non-equilibrium deviation $\mathbf{f} - \mathbf{f}^{\text{eq}}$ into physically meaningful components, you can verify that conservation laws are respected, that transport coefficients are correct, and that numerical artifacts are controlled.

BGK Is a Special Case: All Dials Locked Together

Here is the crucial connection. Set $\mathbf{S} = \frac{\Delta t}{\tau}\mathbf{I}$ (the identity matrix times a single rate). Then the moment-space collision (17.43) becomes:

$$\mathbf{m}^* = \mathbf{m} - \frac{\Delta t}{\tau}(\mathbf{m} - \mathbf{m}^{\text{eq}})$$

and the full round trip is:

$$\mathbf{f}^* = \mathbf{M}^{-1}\left[\mathbf{M}\mathbf{f} - \frac{\Delta t}{\tau}\mathbf{M}\left(\mathbf{f} - \mathbf{f}^{\text{eq}}\right)\right] = \mathbf{f} - \frac{\Delta t}{\tau}(\mathbf{f} - \mathbf{f}^{\text{eq}})$$

The $\mathbf{M}^{-1}$ and $\mathbf{M}$ cancel. We recover exactly the BGK equation (17.32). **BGK was always operating in moment space** — it just didn't need to make the round trip explicit because every moment was being relaxed at the same rate. The single τ locks all the dials together: conservation, transport, and ghost moments all receive the same treatment. This is why BGK works for isotropic diffusion: when $s_1 = s_2$, the x- and y-flux moments relax at the same rate, producing equal diffusion in all directions. And this is why BGK *cannot* produce anisotropic diffusion: there is no way to set $s_1 \neq s_2$ when $\mathbf{S}$ is constrained to be a scalar multiple of the identity.

⚠ Key Insight

Conservation in BGK — a hidden accident: In BGK, the conserved moment (density) formally receives the same relaxation rate $s_0 = \Delta t/\tau$ as every other moment. But conservation is not violated — because f^{eq} is designed so that $m_0^{\text{eq}} = m_0$ (the equilibrium density equals the actual density), the relaxation step computes $m_0^* = m_0 - \frac{\Delta t}{\tau}(m_0 - m_0) = m_0$. Conservation holds, but only because the equilibrium was carefully constructed to make it so. In the moment-space view, you enforce conservation *explicitly* by setting $s_0 = 0$ rather than relying on this cancellation.

What Each Moment Sees During Collision

To make this concrete, consider a D2Q5 node mid-simulation. The distributions f_i are some non-equilibrium state (a diffusion front is passing through). When we decode:

- $m_0 = \sum_i f_i = \phi$ — the local voltage. After collision with $s_0 = 0$: $m_0^* = \phi$. Unchanged. Mass conservation enforced.
- $m_1 = \sum_i c_{ix} f_i = J_x$ — the x-flux. Non-zero because a diffusion front creates a gradient. After collision: $m_1^* = J_x - s_1(J_x - 0) = (1 - s_1)J_x$. The flux decays toward zero (equilibrium has no net flux). The rate s_1 determines how fast — which sets the diffusion coefficient D.
- $m_2 = \sum_i c_{iy} f_i = J_y$ — same story for y.
- m_3, m_4 — the ghost moments. They relax too, but toward values dictated by the lattice weights, not by any conservation law. Their rates s_3, s_4 are free parameters.

This decomposition reveals the internal structure that equation (17.32) hides. You can now **verify** conservation (is m_0 really unchanged?), **measure** the flux relaxation rate (does it match your desired D?), and **tune** the ghost moments independently of the physics.

Worked Example: One D2Q5 Collision in Both Spaces

Let us make the round trip concrete with numbers. Consider a D2Q5 node with $\phi = 1.0$, $\Delta x = 0.025$ cm, $\Delta t = 0.01$ ms, and isotropic diffusivity $D = 0.001$ cm^2/ms. From equation (17.29):

$$\tau = \frac{\Delta t}{2} + \frac{3D\Delta t}{\Delta x^2} = 0.005 + \frac{3(0.001)(0.01)}{0.000625} = 0.005 + 0.048 = 0.053 \text{ ms}$$

so $\omega = \Delta t/\tau = 0.01/0.053 \approx 0.189$. Suppose the current distributions are (a diffusion front has created a slight asymmetry):

$$\mathbf{f} = (0.340,\ 0.175,\ 0.155,\ 0.168,\ 0.162)^T$$

Check: $\sum f_i = 1.000 = \phi$. Good. The equilibrium is $f_i^{\text{eq}} = w_i\phi = (1/3,\ 1/6,\ 1/6,\ 1/6,\ 1/6) \approx (0.333,\ 0.167,\ 0.167,\ 0.167,\ 0.167)$. **BGK (distribution space):**

$$f_i^* = f_i - 0.189(f_i - f_i^{\text{eq}})$$

Computing each component:

i	f_i	f_i^{eq}	$f_i - f_i^{\text{eq}}$	f_i^*
0	0.340	0.333	+0.007	0.3387
1	0.175	0.167	+0.008	0.1735
2	0.155	0.167	−0.012	0.1573

3	0.168	0.167	+0.001	0.1678
4	0.162	0.167	−0.005	0.1629

Check: $\sum f_i^* = 1.0002 \approx 1.000$. Conservation holds (the small deviation is rounding). **Moment space (same collision, decoded):** Apply $\mathbf{m} = \mathbf{M}\mathbf{f}$:

k	Moment	m_k	m_k^{eq}	m_k^* $(s_k = \omega)$	Physical role
0	ρ	1.000	1.000	1.000	Conserved — unchanged
1	J_x	+0.020	0	+0.0162	Flux decays → sets D_x
2	J_y	+0.006	0	+0.0049	Flux decays → sets D_y
3	e	0.660	0.667	0.6613	Ghost — no PDE role
4	ϵ	−0.000	0	−0.000	Ghost — no PDE role

Now you can *see* the physics. The density ($m_0 = 1.000$) is untouched — conservation is enforced. The x-flux ($m_1 = 0.020$) relaxes toward zero at rate ω — this sets the diffusion coefficient. The ghost moments relax too, but they don't affect the macroscopic solution. In BGK, all moments happen to use the same ω. In the moment-space view, you can verify each constraint independently. Applying $\mathbf{f}^* = \mathbf{M}^{-1}\mathbf{m}^*$ gives back the same $\mathbf{f}^*$ as the BGK calculation above — the round trip is exact.

Intuition

What we have gained: The moment-space formulation does not change the physics — for BGK with a single τ, the result is identical. What it changes is **visibility**. You can now see which quantity is conserved, which quantities control transport, and which quantities are free parameters with no physical role. This visibility is the foundation for everything that follows: anisotropic diffusion (relaxing x- and y-flux at different rates), stability tuning (adjusting ghost-moment rates without affecting the solution), and the full MRT collision operator.

What Comes Next

We now have the complete theoretical framework for the lattice-Boltzmann method: the thermodynamic foundation (Section 17.1), the Boltzmann equation and BGK simplification (Section 17.2), the discrete lattice-Boltzmann equation (Section 17.3), the equilibrium distribution and its constraints on weights and velocities (Section 17.4), and the moment-space decomposition that reveals the physical content of the collision operator (Section 17.5). In the next chapter, we will put this machinery to work — the f_i are our fundamental variables, and the BGK collision operator relaxes all of them at the same rate $1/\tau$. But there is a more revealing way to look at the same information. In Section 17.5 above, we transformed the distributions into **moment space**, — first by unlocking the moment-space relaxation rates to achieve anisotropic diffusion (Multiple Relaxation Time), then by applying LBM to the cardiac monodomain and bidomain equations.

Intuition

Complete Reference Tables: Appendix A collects the velocity vectors $\mathbf{c}_i$ and weights w_i for all standard lattices (D1Q3, D2Q5, D2Q9, D3Q7, D3Q19, D3Q27) in a single reference table. These are derived from the exactness conditions (17.18)–(17.20) and the lattice geometries described in this chapter.

CHAPTER 18

Lattice-Boltzmann Methods for Monodomain

Chapter 17 built the LBM machinery from first principles: distribution functions, lattice geometry, collision and streaming as fundamental operations, and the τ-D relationship that translates physical diffusion coefficients into lattice parameters. Now we apply that machinery to the cardiac monodomain equation. The key addition is surprisingly simple: an ionic source term that injects stimulus current and ionic feedback into the particle system. This chapter focuses exclusively on this cardiac-specific coupling, the five-stage timestep algorithm, performance trade-offs, and GPU implementation.

18.1 From LBM Theory to Cardiac Practice

Chapter 17 showed that the distribution functions f_i in LBM solve the diffusion equation through collision (local relaxation) and streaming (nearest-neighbor shifts). For the cardiac monodomain equation (equation 7.1), the only new ingredient is the *ionic source term* — the right-hand side of the PDE that couples reaction (ionic model) to diffusion (LBM). This source is simply added to each collision step, distributed among the distributions weighted by w_i.

The core advantage of LBM for cardiac tissue is that it requires no global linear solve. The diffusion substep consists entirely of local operations (collision at each node) and nearest-neighbor shifts (streaming). This is fundamentally different from FEM or implicit RK methods, which assemble and invert large matrices. For GPU computation, where global communication is expensive and fine-grained data locality is prized, this locality is transformative.

18.2 The Ionic Source Term

The monodomain PDE (equation 7.1) couples diffusion and reaction:

$$\chi C_m \frac{\partial V_m}{\partial t} = \nabla \cdot (D \nabla V_m) - (I_{\text{ion}} + I_{\text{stim}})$$

In the operator-splitting framework (Chapter 8), the diffusion and reaction steps are solved separately. For LBM, the diffusion step is handled by the collision and streaming of distributions. The reaction step computes the ionic current I_{ion} from the current voltage and gating variables, and applies the stimulus I_{stim}. The coupling between them occurs through a source term in the collision operator:

(18.1)

$$S = -\frac{I_{\text{ion}} + I_{\text{stim}}}{\chi C_m}$$

This source has units of voltage per unit time (mV/ms). It is injected into the collision step as $\Delta t\, w_i\, S$ distributed to each direction i. This ensures:
(1) The source is spread equally (by weight) across all distributions (isotropy).
(2) The total injected voltage $\sum_i (\Delta t\, w_i\, S) = \Delta t\, S$ is conserved.

Intuition

Physical interpretation. The source S is like a faucet (or drain) at each node, adding (or removing) voltage at a prescribed rate. In reality, this comes from ionic currents flowing across the cell membrane. The LBM distributes this source equally among all particle directions, so the injected voltage contributes equally to diffusion in all cardinal directions. This is correct for isotropic diffusion; anisotropic tissue (fibers) is handled by MRT (Section 18.4).

18.3 BGK Collision with Ionic Coupling

Recall from Chapter 17 (Section 17.3) that the BGK collision operator nudges each distribution toward equilibrium at rate $\omega = 1/\tau$. For cardiac LBM, we simply add the ionic source term:

$$f_i^* = f_i - \omega\,(f_i - w_i\, V) + \Delta t\, w_i\, S \tag{18.2}$$

The first two terms are pure diffusion (the standard BGK relaxation from Chapter 17). The third term injects the ionic source. After collision, streaming (equation 17.15 in Chapter 17) shifts each distribution f_i^* one lattice spacing in direction $\mathbf{e}_i$. No new mathematical machinery is required — it is a straightforward addition of a source term to the relaxation equation.

The τ-D relationship (equation 17.29 in Chapter 17) remains unchanged: $\tau = 0.5 + 3D\Delta t/\Delta x^2$, which governs $\omega = 1/\tau$. Stability is guaranteed for any $\tau > 0.5$, which holds for any positive D.

18.4 MRT for Anisotropic Tissue

Cardiac tissue conducts faster along fibers than across them (1:3–1:4 anisotropy ratio). BGK, which uses a single relaxation rate ω for all directions, produces only isotropic diffusion. To capture anisotropy, we use MRT (Multiple Relaxation Time), introduced in Chapter 17 (Section 17.5).

MRT transforms the distributions into moment space using a matrix $\mathbf{M}$, applies directional relaxation rates τ_x and τ_y (one per axis), then transforms back. For an arbitrary fiber angle θ, the diffusion tensor is rotated:

$$D_{xx} = D_f \cos^2\theta + D_c \sin^2\theta, \quad D_{yy} = D_f \sin^2\theta + D_c \cos^2\theta$$

where D_f and D_c are along-fiber and cross-fiber coefficients. Then τ_x and τ_y are computed from the τ-D relationship (equation 17.29) using D_{xx} and D_{yy} respectively. The full boxed collision equation (with source term) is:

$$\boxed{\mathbf{f}^* = \mathbf{f} - \mathbf{M}^{-1}\,\mathbf{S}\left(\mathbf{M}\mathbf{f} - \mathbf{M}\mathbf{f}^{\mathrm{eq}}\right) + \Delta t\,\mathbf{w}\,S} \tag{18.3}$$

where $\mathbf{S} = \mathrm{diag}(0, 1/\tau_x, 1/\tau_y, 1.0, 1.0)$ for D2Q5, with higher moments set to 1.0 for stability. This is a straightforward extension of the BGK source term — the source vector $\Delta t\,\mathbf{w}\,S$ is appended to the moment-space relaxation, exactly as in BGK. See Chapter 17 (Section 17.5) for the full derivation of the M matrix and moment-space transformation.

18.5 The Complete LBM-EP Timestep

One full timestep of a cardiac LBM simulation integrates both the ionic model (reaction) and the diffusion. Using Godunov splitting (Chapter 8), the ionic step runs first, then diffusion. The five-stage algorithm is the exact sequence the code executes:

Complete LBM-EP step (BGK or MRT): *(18.4)*

$$\begin{aligned}
&\textbf{1. Ionic:} && I_{\mathrm{ion}} = \mathrm{IonicModel.compute_Iion}(V, \mathbf{s}); \quad \text{update } \mathbf{s} \\
&\textbf{2. Source:} && S = -\frac{I_{\mathrm{ion}} + I_{\mathrm{stim}}}{\chi\, C_m} \\
&\textbf{3. Collide:} && f_i^* = f_i - \omega\,(f_i - w_i\,V) + \Delta t\, w_i\, S \quad \text{(or MRT eq. 18.3)} \\
&\textbf{4. Stream:} && f_i(\mathbf{x} + \mathbf{e}_i) \leftarrow f_i^*(\mathbf{x}) \quad \forall i \\
&\textbf{5. Recover:} && V^{n+1} = \textstyle\sum_i f_i \quad \text{(apply bounce-back on boundaries)}
\end{aligned}$$

Steps 3–5 constitute the diffusion solver. Note the absence of matrix assembly, matrix-vector products, or iterative convergence checks. This is why LBM achieves its speed advantage on GPUs.

Intuition

Worked example. For cardiac tissue with $D = 0.001$ cm^2/ms, $\Delta x = 0.025$ cm, $\Delta t = 0.01$ ms: $\tau = 0.5 + 3 \times 0.001 \times 0.01/0.025^2 = 0.548$, so $\omega = 1/0.548 = 1.825$. Consider a node with

$V = -85$ mV, all distributions at equilibrium, and a stimulus $I_{\text{stim}} = -52$ μA/μF (with $I_{\text{ion}} = 0$, $\chi = 1400$ cm^{-1}, $C_m = 1$ μF/cm^2).
Then $S = 52/1400 = 0.0371$ mV/ms.
After collision: $f_0^* = f_0 + 0.01 \times \frac{1}{3} \times 0.0371 \approx f_0 + 1.24 \times 10^{-4}$.
After streaming and recovery: $V^{n+1} = \sum f_i \approx -85 + 0.01 \times 0.0371 = -84.9996$ mV. The stimulus produces a tiny depolarization in one 0.01 ms step, as expected.

18.6 LBM vs. Classical Methods: When to Choose What

LBM, explicit methods (Chapter 9), and implicit methods (Chapter 10) each have distinct trade-offs:

Aspect	Explicit (Ch 9)	Implicit (Ch 10)	LBM-BGK
Linear solve?	No	Yes (PCG/FFT)	No
CFL-limited?	Yes ($\Delta t \leq h^2/2d$)	No	No ($\tau > 0.5$ always)
Anisotropy?	Via matrix	Via matrix	BGK = isotropic; MRT = anisotropic
Unstructured mesh?	Yes (FEM)	Yes (FEM)	No (Cartesian only)
GPU suitability	Good (matvec)	Fair (global reductions)	Excellent (zero sync)
Memory per node	1 float (V)	1 float + sparse $\mathbf{A}$	5–7 floats (f_i)
Typical use	Coarse mesh, small Δt	Fine mesh, larger Δt	GPU throughput, parameter sweeps

When to use LBM: GPU hardware, Cartesian grids, anisotropic tissue (with MRT), and when throughput (simulations per unit time) matters more than memory. The Rapaka et al. (2012) paper on LBM-EP achieved 8.75× speedup on synthetic slabs and 45× on patient-specific geometries, all on a single CPU core. On modern GPUs without any optimization, the advantage is even larger because LBM eliminates the global synchronization points that slow down PCG solvers.

18.7 GPU Acceleration and the Engine V5.4 Implementation

LBM is a textbook example of SIMD parallelism: every node performs the same collision and streaming operations in lockstep. There are no global data dependencies, no reduce operations, no convergence criteria — each GPU thread is completely independent. On modern GPUs, this translates to near-peak memory bandwidth utilization and minimal synchronization overhead.

⚙ Engine V5.4 Connection

In Engine V5.4, the LBM monodomain solver is in the `cardiac_sim.simulation.lbm` package. The lattice definitions (`D2Q5` , `D3Q7`) are frozen dataclasses holding weights, velocity vectors, and the τ-D conversion formula. Collision operators (`BGKCollision` , `MRTCollision`) inherit from `CollisionOperator(ABC)` and implement the update step (equation 18.2 or 18.3). The `LBMState` container holds the distribution tensor $\mathbf{f}$ (shape $Q \times N_x \times N_y$ for 2D), voltage V, ionic states $\mathbf{s}$, and the domain mask. The `LBMSimulation` class orchestrates the five-step loop: call `collision.collide(f, V, source, dt)` , apply `state.stream()` (one `torch.roll` per direction), `state.apply_bounce_back()` , and finally `state.recover_voltage()` to compute $V = \sum f_i$. The ionic substep is identical to the classical path — only the diffusion step changes. Factory functions `create_isotropic_bgk(D, dx, dt)` and `create_anisotropic_mrt(D_fiber, D_cross, fiber_angle, dx, dt)` handle initialization.

With the monodomain LBM now complete, we turn to its greatest challenge: the bidomain system. The parabolic intracellular equation can be solved by LBM, but the elliptic extracellular equation has no time dependence — it is a constraint that must be satisfied at every instant. Chapter 19 develops three strategies to overcome this fundamental obstacle: pseudo-time relaxation, hybrid LBM–classical solvers, and the dual-lattice architecture.

CHAPTER 19

Lattice-Boltzmann Methods for Bidomain

> *Engine V5.4's LBM solver achieves 10–45× speedups over FEM for the monodomain equation. Can this advantage be preserved for bidomain? Yes — but the elliptic equation requires creative treatment. This chapter develops the core strategies: pseudo-time relaxation, hybrid LBM–classical solvers, and the dual-lattice architecture that couples two independent distribution systems. We show how to extend the computational fiction of particle distributions to solve problems with fundamentally different mathematical structure.*

19.1 The Bidomain Challenge: One Parabolic + One Elliptic

Imagine two fish tanks connected by pipes: an intracellular tank (inside the cells) and an extracellular tank (outside the cells). Water flows from one tank to the other through countless tiny pipes — the ion channels. The voltage across the cell membrane is the water-level difference. Now here is the key: the intracellular voltage evolves over time (it changes as currents flow), so it satisfies a time-dependent equation. But the extracellular voltage is constrained to always satisfy a balance — at every instant, whatever current flows out of the intracellular space must flow through the external medium. This constraint is instantaneous, with no time dependence. One tank's level evolves; the other's is a slave variable determined by the constraint.

The bidomain equations (from Chapter 12) are:

$$\chi C_m \frac{\partial V_m}{\partial t} + I_{\text{ion}} = \nabla\cdot(\mathbf{D}_i \nabla V_m) + \nabla\cdot(\mathbf{D}_i \nabla \phi_e)$$

$$0 = \nabla\cdot\left[(\mathbf{D}_i + \mathbf{D}_e)\nabla\phi_e\right] + \nabla\cdot(\mathbf{D}_i \nabla V_m) + I_{\text{stim}}$$

The first equation is *parabolic* — it has a time derivative, so it describes an evolution. The second is *elliptic* — no time derivative at all. It is a constraint that must be satisfied at every instant. LBM, by its nature, is a time-marching method: it evolves distributions forward in time. This is perfect for the parabolic equation but fundamentally at odds with the elliptic constraint.

The core mismatch is this: LBM asks "How do the distributions evolve?" But the elliptic equation says "The distributions must be such that this constraint is satisfied right now." These are two different questions.

Resolving the tension between them — evolution versus constraint — is the central challenge of bidomain LBM.

Intuition

Think of the monodomain equation (Chapter 18) as a single fish tank with a changing water level. LBM marches the level forward in time with each step. The bidomain is two tanks. The intracellular tank (Vm) still evolves. But the extracellular tank (φe) is not free to evolve — its level is pinned down by the requirement that water conservation is always satisfied. This is why φe is an elliptic equation: it is an instantaneous constraint, not a time-evolution equation.

19.2 Strategy 1: Pseudo-Time Relaxation

If you heat a cold metal plate in a furnace with a fixed pattern of heat sources, the temperature rises over time. Eventually it reaches a steady state where the heat input exactly balances the heat loss through the edges. The temperature at that steady state is determined by the heat sources alone — it is independent of the initial condition or the "time" you waited. In mathematics, finding this steady state is the same as solving the elliptic equation that governs heat diffusion in equilibrium.

Pseudo-time relaxation exploits this observation. We cannot use LBM directly on an elliptic equation because there is no time-derivative to march. But we can invent a fictitious time — call it τ (not to be confused with the relaxation time parameter) — and add an artificial time derivative to the elliptic equation:

$$\frac{\partial \phi_e}{\partial \tau} = \nabla \cdot \left[(\mathbf{D}_i + \mathbf{D}_e) \, \nabla \phi_e \right] + \nabla \cdot (\mathbf{D}_i \, \nabla V_m) + I_{\text{stim}} \tag{19.1}$$

This is now a parabolic equation in the pseudo-time τ. We march it with LBM in pseudo-time for many sub-iterations until the solution settles to a steady state, meaning $\partial \phi_e / \partial \tau$ becomes negligibly small. At that point, the solution is the same as what the elliptic equation demands — the constraint is satisfied.

The beauty of this approach: LBM works just as well on the pseudo-time equation as on the physical parabolic equation. The lattice parameter τ_e is set by the same Chapman–Enskog relationship (equation 17.29):

$$\tau_e = \frac{1}{2} + \frac{3 \, (\mathbf{D}_i + \mathbf{D}_e) \, \Delta \tau}{\Delta x^2} \tag{19.2}$$

where $\Delta\tau$ is the pseudo-time step size and the conductivity is $\mathbf{D}_i + \mathbf{D}_e$ (the sum of intracellular and extracellular tensors, as required by the elliptic equation).

Intuition

The convergence speed depends on τ_e (which determines the collision rate) and on the geometry of the domain. A larger τ_e means gentler relaxation and slower convergence. Typical practice is to use a moderately large τ_e (around 0.8–1.5) to balance stability with speed. The number of pseudo-time sub-iterations per physical time step is usually 10–100, depending on the desired residual reduction. With multigrid acceleration (not detailed here), this can be reduced to 5–10 iterations.

19.3 Strategy 2: Hybrid LBM–Classical Solver

You would not use a hammer to write on paper — you use a pen. And you would not use a pen to drive nails — you use a hammer. The principle is simple: use the right tool for the job. LBM is superb at time-marching parabolic equations on Cartesian grids with GPUs. Classical solvers (conjugate gradient with multigrid preconditioning) are superb at solving elliptic systems exactly (or nearly exactly) without iteration over a fictitious time.

The hybrid approach splits the labor:

- **V_m equation (parabolic)**: Use LBM with MRT collision (as in monodomain). Advance it by one physical time step.
- **ϕ_e equation (elliptic)**: Use a classical solver — Conjugate Gradient (CG) preconditioned by Algebraic Multigrid (AMG). Solve it directly to a tight tolerance.

At each physical time step:

1. Compute ionic currents and update gates (Rush–Larsen).
2. March V_m forward one LBM step using the V_m distribution lattice.
3. Assemble the right-hand side for the elliptic ϕ_e equation, which includes the V_m gradient (now known from step 2) and the stimulus term.
4. Solve the elliptic system $\nabla\cdot\left[\left(\mathbf{D}_i + \mathbf{D}_e\right)\nabla\phi_e\right] = \cdots$ using CG+AMG, yielding ϕ_e.
5. Feed ϕ_e back as a source term for the next step's V_m equation.

The key advantage: avoiding the pseudo-time convergence loop. The classical solver reaches machine precision in 10–30 iterations (typically much faster than 100 pseudo-time iterations). The tradeoff: data must be transferred between the LBM lattice (which lives on a Cartesian GPU grid) and the classical solver's matrix assembly (which may involve FEM or FDM operators).

Mathematical Note

The hybrid approach requires careful bookkeeping of the gradient ∇V_m on the LBM lattice and its transfer to the classical solver. One way to do this: compute ∇V_m on the Cartesian grid using finite differences (natural in LBM since we have nearest-neighbor data), then pass this to the classical solver's RHS assembly. The solver can run on the same grid (FDM) or on a finer/coarser grid (via interpolation operators) to trade memory for accuracy.

19.4 The Dual-Lattice Architecture

Instead of mixing tools, use two of the same tool — but in different modes. Imagine having two bookkeepers, each tracking inventory for different tanks. Tank 1's bookkeeper (Lattice 1) evolves the intracellular voltage. Tank 2's bookkeeper (Lattice 2) repeatedly adjusts the extracellular potential until the water-balance constraint is met. They hand notes to each other at the end of each physical time step, telling each other what changed.

The dual-lattice architecture maintains two independent LBM lattices, each solving its own equation:

- **Lattice 1**: Distribution functions $f_i^{(m)}(\mathbf{x}, t)$ solving the parabolic V_m equation. Relaxation time τ_m encodes the intracellular diffusivity $\mathbf{D}_i$.
- **Lattice 2**: Distribution functions $f_i^{(e)}(\mathbf{x}, t)$ solving the pseudo-time-augmented ϕ_e equation. Relaxation time τ_e encodes the sum $\mathbf{D}_i + \mathbf{D}_e$.

The two lattices are coupled through source terms. The source for Lattice 1 includes the gradient of ϕ_e (the extracellular potential) plus the ionic current. The source for Lattice 2 includes the gradient of V_m plus the stimulus:

$$f_i^{(m)*} = f_i^{(m)} - \omega_m \left(f_i^{(m)} - f_i^{(m)\text{eq}}\right) + \Delta t \, w_i \left[\text{ionic term} + \text{coupling to } \phi_e\right] \tag{19.3}$$

$$f_i^{(e)*} = f_i^{(e)} - \omega_e \left(f_i^{(e)} - f_i^{(e)\text{eq}}\right) + \Delta \tau \, w_i \left[\text{coupling to } V_m + \text{stimulus}\right] \tag{19.4}$$

The algorithm at each physical time step is:

Dual-Lattice Bidomain LBM Timestep:

1. Reaction:	Ionic currents and state updates (Rush–Larsen)
2. Vm update:	One LBM step on Lattice 1 (physical time step Δt)
3. φ_e convergence:	Iterate Lattice 2 until $\frac{\partial \phi_e}{\partial \tau}$ is negligible
4. Source exchange:	Compute coupling terms for the next physical step

Intuition

Worked example outline. Suppose at the start of a timestep: V_m is known (say, -85 mV at most nodes), and ϕ_e is at its previous steady state. We compute ionic currents (depends on V_m and gates). Lattice 1 takes one LBM step, advancing V_m to -84 mV (approximately). Now V_m has changed, so the constraint for ϕ_e has changed (because the RHS includes $\nabla \cdot (\mathbf{D}_i \nabla V_m)$). Lattice 2 then iterates in pseudo-time, re-adjusting ϕ_e at every node until the new steady state is reached. This typically takes 10–50 pseudo-time iterations. Once converged, Lattice 1 knows the new ϕ_e value for the next physical step's source term.

How do the three strategies compare?

Architecture	V_m Solver	ϕ_e Solver	Pros	Cons
A: Dual-Lattice	LBM	LBM (pseudo-time)	Pure LBM; maximal GPU parallelism; unified code path	Pseudo-time convergence overhead (typically 20–100 sub-iterations)
B: Hybrid	LBM	CG+AMG	Optimal solver for each PDE type; fewer iterations for ϕ_e	Data transfer overhead between LBM and classical grids
C: Full Classical	FEM/FDM	FEM/FDM	Mature solver ecosystem; handles unstructured meshes	Slower than LBM for parabolic; matrix assembly overhead

Architecture A (dual-lattice) is the most elegant and maximizes GPU utilization. It is the natural choice for research and exploration. Architecture B (hybrid) trades elegance for robustness: it leverages decades of optimization in multigrid solvers and avoids the pseudo-time convergence cost. In practice, Architecture A is recommended as the starting point for Engine V5.4, with Architecture B as a fallback.

⚙ Engine V5.4 Connection

Engine V5.4 already has the infrastructure for Lattice 1: the D3Q7 lattice definitions, the MRT collision operator, and the torch-based streaming and recovery. For Lattice 2, you need: (1) a second set of distribution arrays $f_i^{(e)}$, (2) a pseudo-time loop that iterates until convergence (checking $\|\partial \phi_e / \partial \tau\|$ after

each step), (3) source term construction that couples V_m gradients from Lattice 1 into Lattice 2's collision. The factory functions can be extended: `create_dual_lattice_bidomain(D_i, D_e, dx, dt)` would initialize both lattices and return a `DualLatticeBidomainSolver` object with methods `step_reaction()`, `step_vm()`, `converge_phi_e(residual_tol)`.

19.5 Performance and Implementation Outlook

How much faster is bidomain LBM than classical bidomain solvers? That depends on how much time the classical solver spends on the elliptic solve. If the elliptic solve is fast (because the domain is simple or multigrid works well), then the speedup is modest. If the elliptic solve dominates, then LBM's advantage is large.

Based on the literature for monodomain LBM (which achieves 10–45× speedup over FEM) and accounting for the added cost of the elliptic ϕ_e equation, estimated performance ranges for bidomain LBM are:

- **Dual-Lattice (Architecture A)**: 5–20× speedup over classical FEM bidomain, depending on the pseudo-time convergence rate and domain geometry. The lower end occurs when pseudo-time takes many iterations; the upper end occurs on simple geometries or with multigrid acceleration of the pseudo-time loop.
- **Hybrid (Architecture B)**: 3–10× speedup. The elliptic solve using AMG is already near-optimal, so gains come mainly from the LBM parabolic solve and from avoiding the global reductions (inner products) required by iterative classical solvers.

Memory overhead: Each architecture stores two voltage fields (V_m and ϕ_e) plus either one or two lattices. The dual-lattice uses $2Q$ floats per node (two sets of Q distributions); the hybrid uses Q floats plus the classical solver's matrix storage. For a 3D grid with D3Q7 ($Q = 7$), the dual-lattice costs 14 floats per node — manageable on modern GPUs with 24–80 GB.

The recommended implementation path for Engine V5.4:

1. **Phase 1:** Extend the existing LBM infrastructure to support dual lattices. Test on a simple 2D domain with uniform conductivity.
2. **Phase 2:** Implement pseudo-time convergence checks and convergence acceleration (e.g., line-by-line SOR, incomplete factorization).
3. **Phase 3:** Add the hybrid solver as an alternative path: call external AMG libraries (AMGX on GPU, Hypre on CPU).
4. **Phase 4:** Benchmark against classical FEM bidomain solvers on realistic geometries and compare accuracy and wall-clock time.

5. **Phase 5:** Extend to anisotropic conductivity (different D_f and D_c for intracellular and extracellular spaces) using MRT on both lattices.

⚙ Engine V5.4 Connection

Recommended starting point for Engine V5.4: Implement Architecture A (dual-lattice) first. The pseudocode:

```
class DualLatticeBidomainLBMSolver(ABC):
    def __init__(self, D_i, D_e, dx, dt, ionic_model):
        self.lattice_vm = LBMLattice(D_i, dx, dt)
        self.lattice_phi_e = LBMLattice(D_i + D_e, dx, dt_pseudo)
        self.ionic = ionic_model

    def step(self, V_m, phi_e, I_stim, residual_tol=1e-6):
        # Step 1: Ionic
        I_ion, gates = self.ionic.step(V_m)

        # Step 2: V_m LBM
        source_vm = -(I_ion + I_stim) / (chi * Cm)
        source_vm += gradient_phi_e  # coupling term
        V_m = self.lattice_vm.step(source_vm)

        # Step 3: phi_e convergence (pseudo-time loop)
        source_phi_e = -gradient(D_i @ gradient(V_m)) + I_stim_e
        phi_e = self.lattice_phi_e.converge(source_phi_e, tol=residual_tol)

        return V_m, phi_e, gates
```

Extend `LBMLattice` (already in the codebase for monodomain) to support pseudo-time iteration with convergence checks. Reuse the collision operator; only the time step size and convergence criterion change.

With the completion of Part IV (Chapters 17–19), we have the full mathematical and algorithmic foundation for bidomain electrophysiology. From the ionic model (Part I) through the monodomain and bidomain frameworks (Parts II and III), every piece is in place for a production cardiac simulator. The final part of this textbook turns to validation, performance optimization, and the art of choosing the right solver for your problem — the practical craft of turning equations into fast, accurate simulations.

APPENDIX A

A Visual Guide to Differential Equations

This appendix steps back from the bidomain model to build a deep, visual intuition for the three fundamental types of partial differential equations: parabolic, elliptic, and hyperbolic. We begin with the heat equation — the simplest and most intuitive PDE — and use it as a lens to understand all three families. The style here follows the philosophy of 3Blue1Brown: start with a concrete picture, build understanding layer by layer, and let the math emerge naturally from the intuition.

A.1 The Heat Equation: Where It All Begins

Imagine you have a thin metal rod. You heat one spot in the middle with a blowtorch, creating a sharp temperature peak, and then you turn off the blowtorch. What happens?

You already know the answer intuitively: the hot spot cools down, and the warmth spreads outward into the cooler parts of the rod. Over time, the temperature profile gets smoother and flatter, until eventually the whole rod reaches a uniform temperature. This simple physical process — heat spreading from hot regions to cold regions — is described by one of the most important equations in all of mathematics:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2} \tag{A.1}$$

This is the **heat equation** in one dimension. Here $u(x, t)$ is the temperature at position x and time t, and $\alpha > 0$ is the *thermal diffusivity* — a material property that tells you how quickly heat spreads (copper has high α; wood has low α).

But what does this equation actually *say*? Let us read it like a sentence.

Intuition

Reading the heat equation as English: "The rate at which the temperature changes at a point ($\partial u/\partial t$) is proportional to how much that point's temperature differs from the average of its neighbors ($\partial^2 u/\partial x^2$)." If a point is hotter than its surroundings, it cools down. If it is cooler, it warms up. The equation says nothing more — and nothing less — than this.

This is the fundamental insight, and it is worth dwelling on. The second derivative $\partial^2 u/\partial x^2$ is not just a mathematical abstraction — it measures *concavity*. At a temperature peak (concave down), the second

derivative is negative, so $\partial u/\partial t < 0$: the temperature decreases. At a temperature valley (concave up), the second derivative is positive, so $\partial u/\partial t > 0$: the temperature increases. The heat equation is a machine that relentlessly destroys peaks and fills valleys.

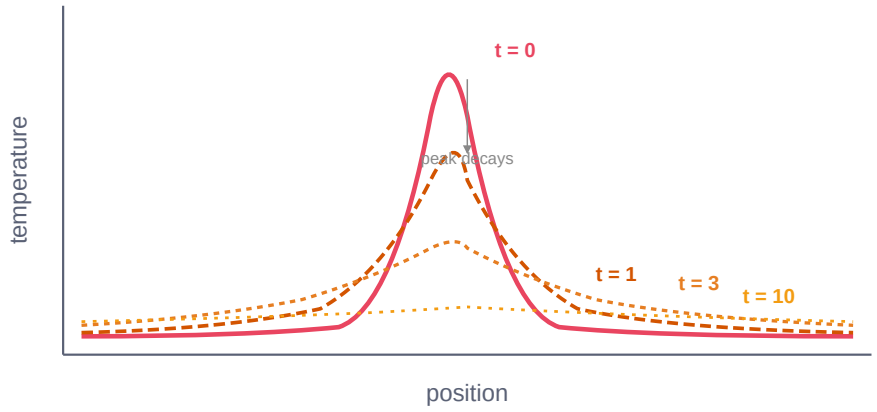


Figure A.1. *A sharp temperature peak (red) diffuses over time, becoming broader and flatter. The heat equation smooths all discontinuities — information about the initial sharp shape is irreversibly lost.*

Notice three properties that are visible even from the picture:

1. **Smoothing.** The solution gets smoother over time. A sharp corner at $t = 0$ becomes an infinitely smooth curve at any $t > 0$. The heat equation is a "smoothing machine."
2. **Irreversibility.** You cannot run time backwards and recover the sharp peak from the flat profile. Information is destroyed. (Mathematically: the backwards heat equation is ill-posed.)
3. **Maximum principle.** The hottest point at any later time is never hotter than the hottest point at $t = 0$. Diffusion can only reduce extremes, never amplify them.

These three properties — smoothing, irreversibility, and a maximum principle — are the *defining fingerprint* of **parabolic** equations. Every parabolic PDE shares them.

A.2 What the Laplacian Really Means

Before we classify PDEs, we need to build intuition for the single most important differential operator in physics: the **Laplacian** $\nabla^2 u$ (also written Δu).

In one dimension, $\nabla^2 u = \partial^2 u/\partial x^2$ — just the second derivative. In two or three dimensions:

$$\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2}$$

But what does it *mean*? Here is the deepest and most useful interpretation:

⚠ Key Insight

The Laplacian measures the difference between a point's value and the average of its neighbors. Specifically, for a function u that is smooth enough:

$$\nabla^2 u(\mathbf{x}) = \lim_{r \to 0} \frac{2d}{r^2} \left[\overline{u}_r(\mathbf{x}) - u(\mathbf{x}) \right]$$

where $\overline{u}_r(\mathbf{x})$ is the average of u on a sphere of radius r centered at $\mathbf{x}$, and d is the spatial dimension. If $u(\mathbf{x})$ is *less* than its neighborhood average, then $\nabla^2 u > 0$. If *more*, then $\nabla^2 u < 0$.

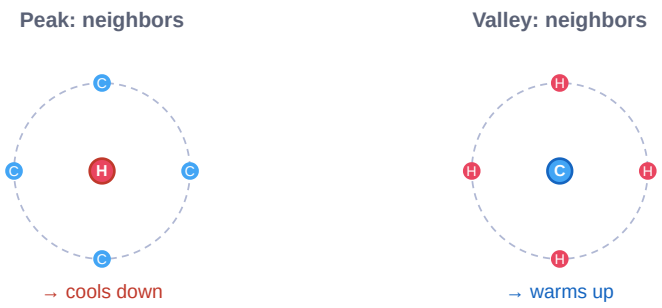


Figure A.2. *The Laplacian as a "disagreement detector." Left: a hot point (H) surrounded by cool neighbors (C) has* $\nabla^2 u < 0$. *Right: a cool point surrounded by hot neighbors has* $\nabla^2 u > 0$. *The heat equation says: resolve the disagreement.*

This is why the Laplacian appears in so many equations across physics. Any time nature wants to smooth out differences — heat, chemical concentration, electrical potential — the Laplacian is doing the work. The heat equation $\partial u / \partial t = \alpha \nabla^2 u$ simply says: *evolve toward the local average*.

Think About It

A discrete analogy. On a grid, approximate $\nabla^2 u$ at node i as $\nabla^2 u_i \approx \frac{1}{h^2}(\bar{u}_{\text{neighbors}} - u_i)$ where $\bar{u}_{\text{neighbors}}$ is the average of the neighboring values. The heat equation on a grid becomes: each node moves a small fraction toward the average of its neighbors at each time step. This is literally a smoothing filter — the same operation used in image processing.

A.3 The Three Families of PDEs

Now we are ready for the big classification. Consider the most general second-order linear PDE in two variables:

$$A\,\frac{\partial^2 u}{\partial x^2} + 2B\,\frac{\partial^2 u}{\partial x\,\partial y} + C\,\frac{\partial^2 u}{\partial y^2} + (\text{lower-order terms}) = 0 \tag{A.2}$$

The character of this equation — its fundamental behavior, the types of solutions it admits, the numerical methods that work — is determined by a single number: the **discriminant**

$$\Delta = B^2 - AC$$

This is exactly the same discriminant from the quadratic formula, and the analogy to conic sections is deep. Just as $B^2 - 4AC$ classifies conics into ellipses, parabolas, and hyperbolas, this discriminant classifies PDEs into three families:

Parabolic ($\Delta = 0$)

Prototype: Heat equation

$$\frac{\partial u}{\partial t} = \alpha\,\nabla^2 u$$

Diffusion, smoothing, irreversible

Elliptic ($\Delta < 0$)

Prototype: Laplace equation

$$\nabla^2 u = 0$$

Equilibrium, steady-state, no time

Hyperbolic ($\Delta > 0$)

Prototype: Wave equation

$$\frac{\partial^2 u}{\partial t^2} = c^2\,\nabla^2 u$$

Propagation, waves, reversible

Think About It

Why does the name "parabolic" come from the degenerate case $\Delta = 0$? Think of the conic $Ax^2 + 2Bxy + Cy^2 = 1$. When $\Delta = 0$, this degenerates from a closed curve (ellipse) to an open, unbounded shape (parabola). Similarly, the PDE loses one "degree of freedom" — information propagates in one direction only (forward in time), just as a parabola is open in one direction.

Let us now build deep intuition for each family.

A.4 Parabolic Equations: Diffusion and Smoothing

The heat equation is the prototype, but parabolic equations describe far more than heat. Any process where "stuff" spreads out from high-concentration regions to low-concentration regions is parabolic: chemical diffusion (Fick's law), financial option pricing (Black–Scholes), probability spreading (Fokker–Planck), and — most relevant to us — the evolution of the transmembrane potential V_m in the monodomain equation.

The unifying feature is the structure: **one time derivative balanced against two spatial derivatives**. This mismatch (∂_t vs. ∂_{xx}) dictates everything:

$$\underbrace{\frac{\partial u}{\partial t}}_{\text{1st order in } t} = \alpha \underbrace{\frac{\partial^2 u}{\partial x^2}}_{\text{2nd order in } x}$$

Because there is only a *first* derivative in time, you need only *one* initial condition: $u(x, 0) = u_0(x)$. Time marches in one direction only. There is no "bouncing back." This is why parabolic equations model irreversible processes.

The smoothing property, precisely stated. If you start with any initial condition — even one with sharp corners or discontinuities — the solution $u(x, t)$ is infinitely smooth for all $t > 0$. This is remarkable. A square-wave initial condition instantly (at any positive time, no matter how small) becomes a C^∞ function. Intuitively: diffusion smears out all sharp features immediately.

Infinite propagation speed. Here is a subtle and often confusing point. Heat propagates at *infinite speed* in the mathematical heat equation — if you heat one end of an infinitely long rod, the other end feels it instantly (though the effect is exponentially small). This is physically unrealistic but mathematically convenient. The parabolic monodomain equation inherits this property: a stimulus at one point technically affects the entire domain immediately, though the practical effect dies off exponentially fast.

> ⚠ **Key Insight**
>
> **The parabolic CFL condition.** When you discretize a parabolic equation with an explicit time stepper (like forward Euler), stability demands $\Delta t \leq C\,\Delta x^2$. The *quadratic* dependence on Δx is severe: halving the mesh spacing requires quartering the time step. This is why implicit methods (Crank–Nicolson, BDF, SDIRK) are essential for parabolic problems — they are unconditionally stable, allowing Δt dictated by accuracy rather than stability.

A.5 Elliptic Equations: Equilibrium and Constraints

Now remove time from the picture entirely. Ask: *what does the temperature look like after the system has fully equilibrated?* At equilibrium, $\partial u/\partial t = 0$, and the heat equation becomes:

$$\nabla^2 u = 0 \qquad \text{(Laplace's equation)} \tag{A.3}$$

This is the prototypical **elliptic** equation. It says: at every point, the value of u equals the average of its neighbors. There is no time. There is no evolution. It is a pure *constraint*.

Intuition

The rubber sheet analogy. Imagine a flat rubber membrane stretched over a wire frame. The frame is bent into some shape (the boundary condition). Where does the membrane settle? At every interior point, the height of the membrane equals the average height of the surrounding points — because the elastic forces balance in all directions. The equilibrium shape satisfies Laplace's equation.

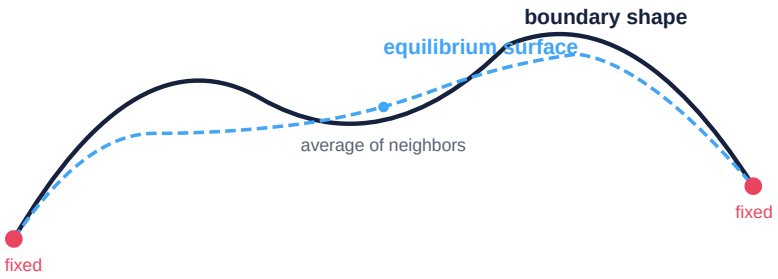


Figure A.3. *The rubber sheet analogy for Laplace's equation. The boundary conditions (wire frame) determine the shape; the interior settles into an equilibrium where each point's value equals the average of its neighbors.*

The key properties of elliptic equations mirror the equilibrium intuition:

1. **Mean value property.** The value at any interior point equals the average over any surrounding sphere. This is the deepest characterization of harmonic functions (solutions to $\nabla^2 u = 0$).
2. **Maximum principle.** The maximum and minimum of u occur on the *boundary*, never in the interior. The rubber sheet cannot bulge above or below the wire frame.
3. **Global coupling.** Changing the boundary condition at one point affects the solution *everywhere* in the interior. Information propagates in all directions simultaneously. There is no concept of "marching" from one end to the other.
4. **No time, no initial conditions.** You need only boundary conditions. The equation determines the interior values uniquely from the boundary.

Think About It

Why you cannot "march" an elliptic equation. In a parabolic equation, you know u at $t = 0$ and march forward to $t = \Delta t$, then to $t = 2\Delta t$, and so on. In an elliptic equation, there is no "forward." Changing

one boundary value ripples through the *entire* domain simultaneously. This is why elliptic equations always require solving a **global linear system** — you must find all unknowns at once.

If there is a source term, Laplace's equation becomes **Poisson's equation**: $\nabla^2 u = f$. This is still elliptic. It says: the difference between a point's value and its neighborhood average is proportional to the local source strength f. This is exactly the form of the bidomain elliptic constraint.

A.6 Hyperbolic Equations: Waves and Propagation

The third family is fundamentally different from the first two. Consider the **wave equation**:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2} \tag{A.4}$$

Notice the crucial structural difference: the *second* derivative in time. Where the heat equation had ∂_t (first order), the wave equation has ∂_{tt} (second order). This single change transforms the physics completely.

With a first-order time derivative, you need one initial condition (u at $t = 0$). With a second-order time derivative, you need *two*: the initial shape $u(x, 0)$ and the initial velocity $\partial u/\partial t|_{t=0}$. This is like specifying both the position and velocity of a particle — and it means the equation is *reversible*. You can run time forward or backward and get valid solutions.

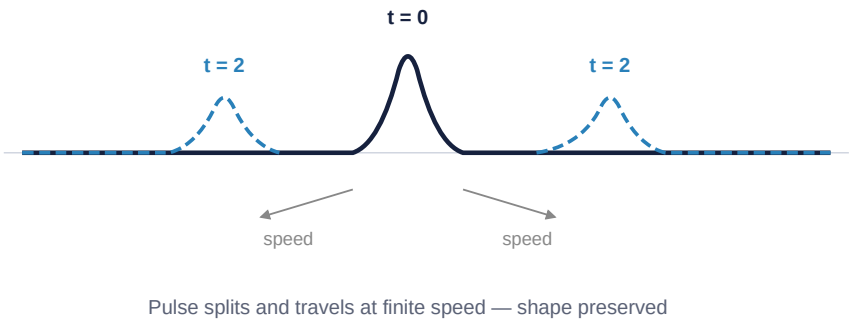


Figure A.4. *The wave equation splits an initial pulse into two copies traveling in opposite directions at speed c. Unlike diffusion, the pulse shape is preserved — no smoothing occurs.*

The contrast with the heat equation could not be sharper:

Property	Parabolic (heat)	Hyperbolic (wave)
Time derivative order	1st (∂_t)	2nd (∂_{tt})

Initial conditions needed	1 (value)	2 (value + velocity)
Reversible?	No	Yes
Propagation speed	Infinite (exponential decay)	Finite (c)
Smoothing?	Yes — destroys sharp features	No — preserves sharp features
Energy	Dissipated over time	Conserved

The finite propagation speed is the hallmark of hyperbolic equations. A disturbance at point x_0 at time t_0 can only be felt at point x after time $(x - x_0)/c$. This region of influence is called the **light cone** (even for non-relativistic waves), and the boundary of this cone consists of **characteristics** — curves in space-time along which information travels.

Intuition

Why cardiac EP is NOT hyperbolic. At first glance, the action potential "wavefront" sweeping through the heart looks like a traveling wave — so you might expect a wave equation. But it is not a wave in the hyperbolic sense. The action potential front is driven by *diffusion* (current spreading from depolarized cells to resting cells) plus a *nonlinear reaction* (ion channels opening in a cascade). There is no second time derivative. There is no energy conservation. And the wavefront *does* smooth out over time. The monodomain and bidomain equations are parabolic, not hyperbolic — the traveling wavefront emerges from the interplay of diffusion and reaction, not from wave mechanics.

A.7 Why the Bidomain Model Has Two Types

We can now understand the bidomain model's structure at a much deeper level. The bidomain system is not a single PDE — it is a *coupled pair*, and each equation belongs to a different family:

$$\underbrace{\chi C_m \frac{\partial V_m}{\partial t} = -\chi\, I_{\text{ion}} + \nabla \cdot (\mathbf{D}_i\, \nabla V_m) + \nabla \cdot (\mathbf{D}_i\, \nabla \phi_e)}_{\text{Parabolic: has } \partial V_m/\partial t} \tag{A.5}$$

$$\underbrace{0 = \nabla \cdot (\mathbf{D}_i\, \nabla V_m) + \nabla \cdot \left[(\mathbf{D}_i + \mathbf{D}_e)\, \nabla \phi_e\right]}_{\text{Elliptic: no time derivative at all}} \tag{A.6}$$

This mixed parabolic–elliptic structure is the source of both the model's physical richness and its computational difficulty:

1. **The parabolic equation** governs the evolution of the transmembrane potential V_m. It is a diffusion equation — with all the properties we discussed: smoothing, irreversibility, CFL restrictions for explicit methods. The "stuff" being diffused is electrical charge, and the diffusion coefficient is determined by the intracellular conductivity tensor $\mathbf{D}_i$. It can in principle be marched forward in time.
2. **The elliptic equation** constrains the extracellular potential ϕ_e. It has no time derivative — it is an *equilibrium condition* that must hold at every instant, like the rubber sheet analogy. At every moment, the extracellular potential adjusts *instantly* so that the total current in and out of every point balances. It cannot be marched — it must be solved as a global linear system.

> ⚠ **Key Insight**
>
> **The fundamental computational consequence.** The parabolic equation can (in theory) be treated explicitly. The elliptic equation *cannot* — ever. This is why every bidomain solver requires at least one implicit linear solve per time step: the elliptic constraint demands it. This is not a choice of method; it is a consequence of the equation's type. No clever algorithm can avoid it, because the elliptic equation couples all spatial points simultaneously.

The monodomain equation, by contrast, is purely parabolic — a single equation with a single time derivative. This is why it is so much cheaper: you never need to solve an elliptic constraint. The LBM methods in Chapter 17 exploit this: a lattice-Boltzmann scheme can handle a parabolic PDE naturally (the collision-streaming cycle is essentially an explicit time stepper), but it has no mechanism for enforcing a global elliptic constraint.

Understanding the type of each equation also tells you *which numerical methods to reach for*:

Equation type	Natural solvers	Avoid
Parabolic (V_m)	Implicit time steppers (CN, BDF, SDIRK), operator splitting, IMEX, LBM	Fully explicit on fine meshes (CFL: $\Delta t \propto \Delta x^2$)
Elliptic (ϕ_e)	Direct solvers, CG/MINRES with AMG preconditioner, multigrid	Explicit time stepping (impossible — no time derivative)

The classification of PDEs into parabolic, elliptic, and hyperbolic is not merely academic taxonomy. It determines which initial/boundary conditions are needed, which numerical methods will work, what stability restrictions apply, and whether information flows forward only or in all directions simultaneously. For the bidomain model, this classification is the key to understanding why the computational challenge is

fundamentally harder than monodomain — and why the solver strategies of Chapter 15 are not optional refinements but structural necessities.

The treatment in this appendix follows the pedagogical philosophy of Grant Sanderson's *3Blue1Brown* series on differential equations. For animated visualizations of these concepts, the reader is encouraged to watch the "Differential Equations" playlist on the 3Blue1Brown YouTube channel.